Computer Networks (2025-2026)
Part 5: Link Layer

Jeroen Famaey
jeroen.famaey@uantwerpen.be

University of Antwerp
Faculty of Science

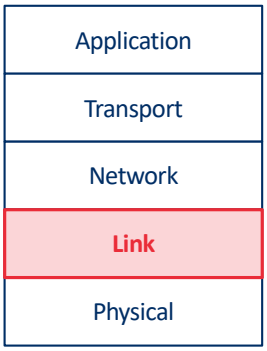
We learned that the network layer provides a communication service between *any* two network hosts. Between the two hosts, datagrams travel over a series of communication links, some wired and some wireless, starting at the source host, passing through a series of packet switches (switches and routers) and ending at the destination host. As we continue down the protocol stack, from the network layer to the link layer, we naturally wonder how packets are sent across the *individual links* that make up the end-to-end communication path. How are the network-layer datagrams encapsulated in the link-layer frames for transmission over a single link? Are different link-layer protocols used in the different links along the communication path? How are transmission conflicts in broadcast links resolved? Is there addressing at the link layer and, if so, how does the link-layer addressing operate with the network-layer addressing? And what exactly is the difference between a switch and a router?

In discussing the link layer, we'll see that there are two fundamentally different types of link-layer channels. The **first type** are broadcast channels, which connect multiple hosts in wireless LANs, in satellite networks, and in hybrid fiber-coaxial cable (HFC) access networks. Since many hosts are connected to the same broadcast communication channel, a so-called medium access protocol is needed to coordinate frame transmission. In some cases, a central controller may be used to coordinate transmissions; in other cases, the hosts themselves coordinate transmissions. The **second type** of link-layer channel is the point-to-point communication link, such as that often found between two routers connected by a long-distance link, or between a user's office computer and the nearby Ethernet switch to which it is connected. Coordinating access to a point-to-point link is simpler.

Table of contents

Introduction to the Link Layer

Link layer overview



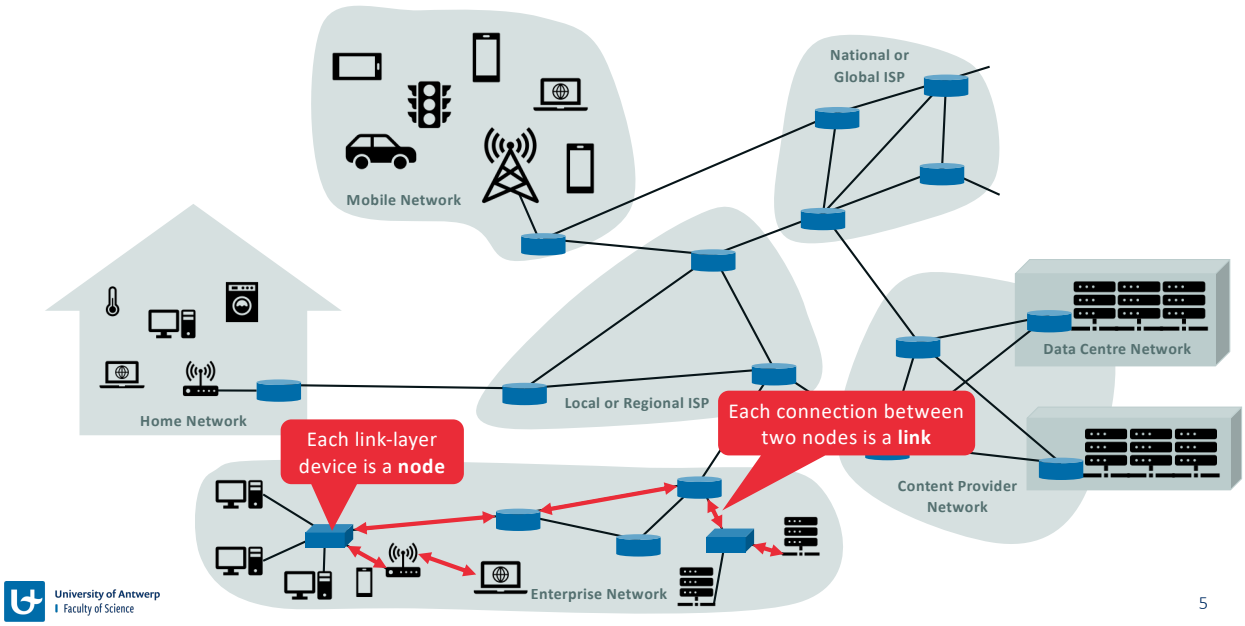
Link Layer

- Responsible for transferring data across a single link (i.e., between 2 nodes)
- Link-layer packets are called **frames**
- Functionality depends on the specific link-layer protocol
  - Reliability
  - Error detection
  - Addressing
  - Media access
  - …
- Examples
  - Ethernet
  - Wi-Fi
  - DOCSIS (Data Over Cable Service Interface Specification)
  - PPP (Point-to-Point Protocol)

To move a packet from one node (host or router) to the next node in the route, the network layer relies on the services of the link layer. In particular, at each node, the network layer passes the datagram down to the link layer, which delivers the datagram to the next node along the route. At this next node, the link layer passes the datagram up to the network layer.

The services provided by the link layer depend on the specific link-layer protocol that is employed over the link. For example, some link-layer protocols provide reliable delivery, from transmitting node, over one link, to receiving node. Note that this reliable delivery service is different from the reliable delivery service of TCP, which provides reliable delivery from one end system to another. Examples of link-layer protocols include Ethernet, WiFi, and the cable access network's DOCSIS protocol. As datagrams typically need to traverse several links to travel from source to destination, a datagram may be handled by different link-layer protocols at different links along its route. For example, a datagram may be handled by Ethernet on one link and by PPP on the next link. The network layer will receive a different service from each of the different link-layer protocols. In this book, we'll refer to the link-layer packets as **frames**.

The link layer enables transmission of data over a single link



Let's begin with some important terminology. We'll find it convenient in this chapter to refer to any device that runs a link-layer (i.e., layer 2) protocol as a **node**. Nodes include hosts, routers, switches, and WiFi access points. We will also refer to the communication channels that connect adjacent nodes along the communication path as **links**. In order for a datagram to be transferred from source host to destination host, it must be moved over each of the *individual links* in the end-to-end path.

As an example, in the enterprise network shown at the bottom of the figure, consider sending a datagram from one of the wireless hosts to one of the servers. This datagram will actually pass through six links: a Wi-Fi link between sending host and Wi-Fi access point, an Ethernet link between the access point and a link-layer switch; a link between the link-layer switch and the router, a link between the two routers; an Ethernet link between the router and a link-layer switch; and finally an Ethernet link between the switch and the server. Over a given link, a transmitting node encapsulates the datagram in a **link-layer frame** and transmits the frame into the link.

Link layer services

Possible functionality offered by different link layer protocols:

- **Framing**: Encapsulate each network-layer datagram in a link-layer frame (including header fields)
- **Link access**: Medium access control (MAC) protocol that specifies the rules of frame transmissions
- **Reliable delivery**: Guarantee to move each network-layer datagram across the link error-free
- **Error detection and correction**: Detecting and/or correcting bit errors due to attenuation or noise

Although the basic service of any link layer is to move a datagram from one node to an adjacent node over a single communication link, the details of the provided service can vary from one link-layer protocol to the next. Possible services that can be offered by a link-layer protocol include:

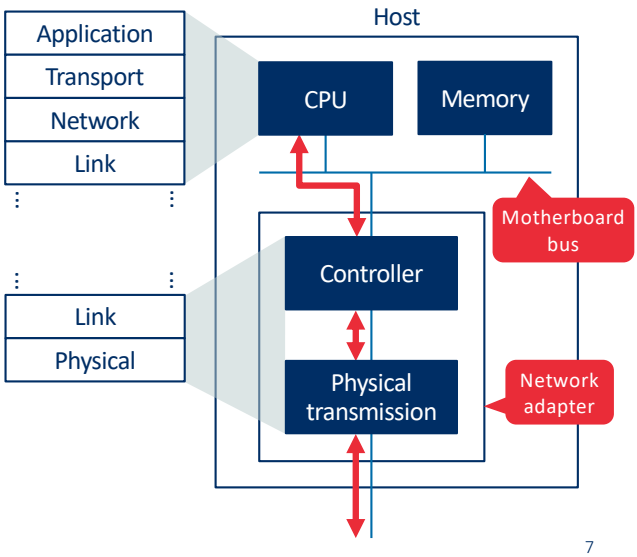
- *Framing.* Almost all link-layer protocols encapsulate each network-layer datagram within a link-layer frame before transmission over the link. A frame consists of a data field, in which the network-layer datagram is inserted, and a number of header fields. The structure of the frame is specified by the link-layer protocol.
- *Link access.* A medium access control (MAC) protocol specifies the rules by which a frame is transmitted onto the link. For point-to-point links that have a single sender at one end of the link and a single receiver at the other end of the link, the MAC protocol is simple (or non-existent)—the sender can send a frame whenever the link is idle. The more interesting case is when multiple nodes share a single broadcast link—the so-called multiple access problem. Here, the MAC protocol serves to coordinate the frame transmissions of the many nodes.
- *Reliable delivery.* When a link-layer protocol provides reliable delivery service, it guarantees to move each network-layer datagram across the link without error. Recall that certain transport-layer protocols (such as TCP) also provide a reliable delivery service. Similar to a transport-layer reliable delivery service, a link-layer reliable delivery service can be achieved with acknowledgments and retransmissions. A link-layer reliable delivery service is often used for links that are prone to high error rates, such as a wireless link, with the goal of correcting an error locally—on the link where the error occurs—rather than forcing an end-to-end retransmission of the data by a transport- or application-layer protocol. However, link-layer reliable delivery can be considered an

unnecessary overhead for low bit-error links, including fiber, coax, and many twisted-pair copper links. For this reason, many wired link-layer protocols do not provide a reliable delivery service.

- *Error detection and correction.* The link-layer hardware in a receiving node can incorrectly decide that a bit in a frame is zero when it was transmitted as a one, and vice versa. Such bit errors are introduced by signal attenuation and electromagnetic noise. Because there is no need to forward a datagram that has an error, many link-layer protocols provide a mechanism to detect such bit errors. This is done by having the transmitting node include error-detection bits in the frame, and having the receiving node perform an error check. Recall that the Internet's transport layer and network layer also provide a limited form of error detection—the Internet checksum. Error detection in the link layer is usually more sophisticated and is implemented in hardware. Error correction is similar to error detection, except that a receiver not only detects when bit errors have occurred in the frame but also determines exactly where in the frame the errors have occurred (and then corrects these errors).

Link layer implementation

- Many link layer functionalities implemented in network adapter hardware:
  - Framing
  - Error detection
  - ...
- Higher-level link-layer functionality implemented in software:
  - Link-layer addressing
  - Activating the network adapter
  - Frame reception interrupts
  - Handling error conditions
  - ...



University of Antwerp
Faculty of Science

7

For the most part, the link layer is implemented on a chip called the **network adapter**, also sometimes known as a **network interface controller (NIC)**. The network adapter implements many link layer services including framing, link access, error detection, and so on. Thus, much of a link-layer controller's functionality is implemented in hardware.

On the sending side, the controller takes a datagram that has been created and stored in host memory by the higher layers of the protocol stack, encapsulates the datagram in a link-layer frame (filling in the frame's various fields), and then transmits the frame into the communication link, following the link-access protocol. On the receiving side, a controller receives the entire frame, and extracts the network-layer datagram. If the link layer performs error detection, then it is the sending controller that sets the error-detection bits in the frame header and it is the receiving controller that performs error detection.

While most of the link layer is implemented in hardware, part of the link layer is implemented in software that runs on the host's CPU. The software components of the link layer implement higher-level link-layer functionality such as assembling link-layer addressing information and activating the controller hardware. On the receiving side, link-layer software responds to controller interrupts (for example, due to the receipt of one or more frames), handling error conditions and passing a datagram up to the network layer. Thus, the link layer is a combination of hardware and software—the place in the protocol stack where software meets hardware.

Error-detection and -correction

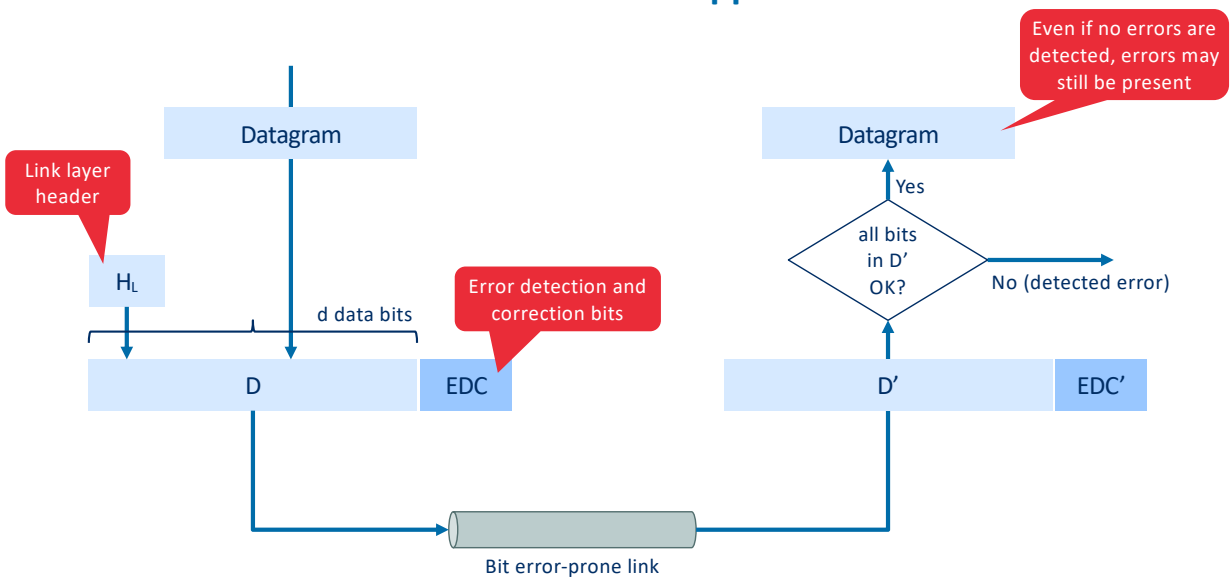
University of Antwerp
Faculty of Science

Computer Networking A Top-Down Approach

Section 6.2

In this part, we'll examine a few of the simplest techniques that can be used to detect and, in some cases, correct such bit errors. A full treatment of the theory and implementation of this topic is itself the topic of many textbooks, and our treatment here is necessarily brief. Our goal here is to develop an intuitive feel for the capabilities that error-detection and -correction techniques provide and to see how a few simple techniques work and are used in practice in the link layer.

General error-detection and –correction approach



University of Antwerp
Faculty of Science

9

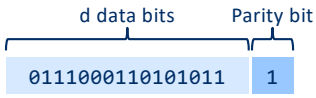
At the sending node, data, *D*, to be protected against bit errors is augmented with error-detection and -correction bits (*EDC*). Typically, the data to be protected includes not only the datagram passed down from the network layer for transmission across the link, but also link-level addressing information, sequence numbers, and other fields in the link frame header. Both *D* and *EDC* are sent to the receiving node in a link-level frame. At the receiving node, a sequence of bits, *D* and *EDC* is received. Note that *D* and *EDC* may differ from the original *D* and *EDC* as a result of in-transit bit flips.

The receiver's challenge is to determine whether or not *D* is the same as the original *D*, given that it has only received *D* and *EDC*. The exact wording of the receiver's decision (we ask whether an error is detected, not whether an error has occurred!) is important. Error-detection and -correction techniques allow the receiver to sometimes, *but not always*, detect that bit errors have occurred. Even with the use of error-detection bits there still may be **undetected bit errors**; that is, the receiver may be unaware that the received information contains bit errors. As a consequence, the receiver might deliver a corrupted datagram to the network layer, or be unaware that the contents of a field in the frame's header has been corrupted. We thus want to choose an error-detection scheme that keeps the probability of such occurrences small. Generally, more sophisticated error-detection and –correction techniques (that is, those that have a smaller probability of allowing undetected bit errors) incur a larger overhead—more computation is needed to compute and transmit a larger number of error-detection and -correction bits.

Parity checks

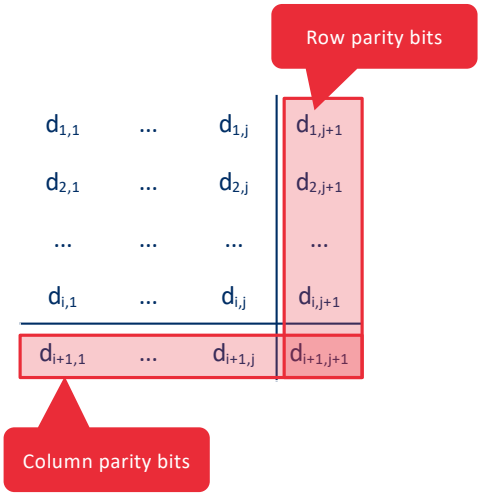
Parity bit

- Add a single error detection bit so that the sum of all d + 1 bits is always even (or odd)
- It can only detect an odd number of bit errors (an even number of errors would go undetected)
- Doesn't work well in practice, as bit-errors often occur in bursts (i.e., they are not independent)
- Example (even parity scheme):



Perhaps the simplest form of error detection is the use of a single **parity bit**. Suppose that the information to be sent, *D*, has *d* bits. In an even parity scheme, the sender simply includes one additional bit and chooses its value such that the total number of 1s in the *d* + 1 bits (the original information plus a parity bit) is even. For odd parity schemes, the parity bit value is chosen such that there is an odd number of 1s. Receiver operation is also simple with a single parity bit. The receiver need only count the number of 1s in the received *d* + 1 bits. If an odd number of 1-valued bits are found with an even parity scheme, the receiver knows that at least one bit error has occurred. More precisely, it knows that some *odd* number of bit errors have occurred. But what happens if an even number of bit errors occur? You should convince yourself that this would result in an undetected error. If the probability of bit errors is small and errors can be assumed to occur independently from one bit to the next, the probability of multiple bit errors in a packet would be extremely small. In this case, a single parity bit might suffice. However, measurements have shown that, rather than occurring independently, errors are often clustered together in "bursts." Under burst error conditions, the probability of undetected errors in a frame protected by single-bit parity can approach 50 percent. Clearly, a more robust error-detection scheme is needed (and, fortunately, is used in practice!).

Two-dimensional generalization of parity scheme



- The *d* bits in D are divided in *i* rows and *j* columns
- A parity value is computed for each row and column
- Single bit errors will show an error in both the row and column parity bit, and can thus be corrected
  - The ability to correct errors is known as **Forward Error Correction (FEC)**

University of Antwerp
Faculty of Science

11

Let's consider a two-dimensional generalization of the single-bit parity scheme. Here, the *d* bits in *D* are divided into *i* rows and *j* columns. A parity value is computed for each row and for each column. The resulting $i + j + 1$ parity bits comprise the link-layer frame's error-detection bits. Suppose now that a single bit error occurs in the original *d* bits of information. With this **two-dimensional parity** scheme, the parity of both the column and the row containing the flipped bit will be in error. The receiver can thus not only *detect* the fact that a single bit error has occurred, but can use the column and row indices of the column and row with parity errors to actually identify the bit that was corrupted and *correct* that error! The ability of the receiver to both detect and correct errors is known as **forward error correction (FEC)**.

Exercise: Correcting errors using the two-dimensional parity scheme

Exercise: Find and correct the single bit-error in the below datagram D'.

- Received datagram D' and row/column parity bits (assuming **even** parity checks):

						Row parity bits ↓
	1	0	1	0	1	1
	1	0	1	1	0	0
	0	1	1	1	0	1
Column parity bits →	0	0	1	0	1	0

Checksumming

- The *d* bits of data are split into k-bit integers, and the sum of these integers is used for error detection
- The **Internet checksum** is based on this approach (RFC 1071)
- As discussed in Part 3, checksums have a lot of limitations, but are easy to calculate (in software)

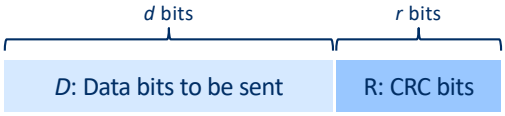
- A brief reminder on the Internet checksum approach:
  - Sender calculates the 1s complement of the sum of all 16-bit words in the UDP/TCP segment (+ pseudo IP header)
  - Receiver adds all 16-bit words of the segment, pseudo header and checksum together and checks if it is 11111111 11111111
  - At network layer, IP does a similar check, but only on the IP header

In checksumming techniques, the *d* bits of data are treated as a sequence of *k*-bit integers. One simple checksumming method is to simply sum these *k*-bit integers and use the resulting sum as the error-detection bits. The **Internet checksum** is based on this approach—bytes of data are treated as 16-bit integers and summed. The 1s complement of this sum then forms the Internet checksum that is carried in the segment header. The receiver checks the checksum by taking the 1s complement of the sum of the received data (including the checksum) and checking whether the result is all 0 bits. If any of the bits are 1, an error is indicated. RFC 1071 discusses the Internet checksum algorithm and its implementation in detail. In the TCP and UDP protocols, the Internet checksum is computed over all fields (header and data fields included). In IP, the checksum is computed over the IP header (since the UDP or TCP segment has its own checksum). In other protocols, for example, XTP, one checksum is computed over the header and another checksum is computed over the entire packet.

Checksumming methods require relatively little packet overhead. For example, the checksums in TCP and UDP use only 16 bits. However, they provide relatively weak protection against errors as compared with cyclic redundancy check (CRC). A natural question at this point is, Why is checksumming used at the transport layer and cyclic redundancy check used at the link layer? Recall that the transport layer is typically implemented in software in a host as part of the host's operating system. Because transport-layer error detection is implemented in software, it is important to have a simple and fast error-detection scheme such as checksumming. On the other hand, error detection at the link layer is implemented in dedicated hardware in adapters, which can rapidly perform the more complex CRC operations.

Cyclic redundancy check (CRC) codes

- Widely used in today's computer networks (and especially link layer protocols)
- Based on polynomial arithmetic
  - Each bit string can be represented as a polynomial with binary coefficients (e.g., 1011 equals x^3+x+1)
  - All calculations are done in modulo-2 arithmetic without carry in addition or borrows in subtraction
  - Addition and subtraction are thus identical and equal to the bitwise exclusive-OR (XOR, or $\oplus$) operation
  - Multiplication by 2^k can be calculated by left shifting a bit pattern by *k* places
- Approach
  - Sender and receiver agree on an r + 1 bit pattern, known as a **generator *G***
  - Sender appends r bits *R* to the data *D* so that the resulting *d + r* bit pattern is divisible by *G*
  - The receiver divides the *d + r* received bits by *G*, if the remainder is non-zero an error has occurred



$D \cdot 2^r \oplus R$

University of Antwerp
Faculty of Science

15

An error-detection technique used widely in today's computer networks is based on **cyclic redundancy check (CRC) codes**. CRC codes are also known as **polynomial codes**, since it is possible to view the bit string to be sent as a polynomial whose coefficients are the 0 and 1 values in the bit string, with operations on the bit string interpreted as polynomial arithmetic.

CRC codes operate as follows. Consider the *d*-bit piece of data, *D*, that the sending node wants to send to the receiving node. The sender and receiver must first agree on an *r* + 1 bit pattern, known as a **generator**, which we will denote as *G*. We will require that the most significant (leftmost) bit of *G* be a 1. For a given piece of data, *D*, the sender will choose *r* additional bits, *R*, and append them to *D* such that the resulting *d* + *r* bit pattern (interpreted as a binary number) is exactly divisible by *G* (i.e., has no remainder) using modulo-2 arithmetic. The process of error checking with CRCs is thus simple: The receiver divides the *d* + *r* received bits by *G*. If the remainder is nonzero, the receiver knows that an error has occurred; otherwise the data is accepted as being correct.

All CRC calculations are done in modulo-2 arithmetic without carries in addition or borrows in subtraction. This means that addition and subtraction are identical, and both are equivalent to the bitwise exclusive-or (XOR) of the operands. Multiplication and division are the same as in base-2 arithmetic, except that any required addition or subtraction is done without carries or borrows. As in regular binary arithmetic, multiplication by 2*k* left shifts a bit pattern by *k* places. Thus, given *D* and *R*, the quantity *D* # 2*r* XOR *R* yields the *d* + *r* bit pattern.

CRC: How does the sender compute R?

- The *d + r* bit pattern should be divisible by *G*, or:

$$D \cdot 2^r \oplus R = nG$$

- If we XOR both sides with *R*, we get:

$$D \cdot 2^r = nG \oplus R$$

- We can thus calculate *R* as:

$$R = \text{remainder}\left(\frac{D \cdot 2^r}{G}\right)$$

University of Antwerp
Faculty of Science

Let us now turn to the crucial question of how the sender computes *R*. Recall that we want to find *R* such that there is an *n* such that

$$D \cdot 2^r \oplus R = nG$$

That is, we want to choose *R* such that *G* divides into $D \cdot 2^r \oplus R$ without remainder. If we XOR (that is, add modulo-2, without carry) *R* to both sides of the above equation, we get

$$D \cdot 2^r = nG \oplus R$$

This equation tells us that if we divide $D \cdot 2^r$ by *G*, the value of the remainder is precisely *R*. In other words, we can calculate *R* as

$$R = \text{remainder}\left(\frac{D \cdot 2^r}{G}\right)$$

Exercise: Calculate the CRC code R for a given datagram D

Exercise: Calculate the CRC bits *R* for *D* = 101110, *G* = 1001, and *r* = 3.

Given:

$$R = \text{remainder}\left(\frac{D \cdot 2^r}{G}\right)$$

Homework: CRC validation

Exercise: Assume r = 3, G = 1001, and a 9 bit message (consisting of D + R) 101110011 is received. Did a bit error occur during transmission?

CRC standards

- Standards have been defined for 8-, 12-, 16-, and 32-bit CRC generators
- Several IEEE link layer protocols use the CRC 32-bit standard with generator

$$G_{CRC\text{-}32} = 10000010011000001000111011011 0111$$

- Error detection probability of the standardized CRC generators
  - Can detect up to r - 1 consecutive bit errors
  - Can also detect consecutive bit errors of length greater than r + 1 with a probability equal to $1 - 0.5^r$
  - Can detect any odd number of bit errors

University of Antwerp
Faculty of Science

21

International standards have been defined for 8-, 12-, 16-, and 32-bit generators, *G*. The CRC-32 32-bit standard, which has been adopted in a number of link-level IEEE protocols, uses a generator of

$$G_{CRC\text{-}32} = 10000010011000001000111011011 0111$$

Each of the CRC standards can detect burst errors of fewer than $r + 1$ bits. (This means that all consecutive bit errors of r bits or fewer will be detected.) Furthermore, under appropriate assumptions, a burst of length greater than $r + 1$ bits is detected with probability $1 - 0.5^r$. Also, each of the CRC standards can detect any odd number of bit errors.

Multiple access links and protocols

University of Antwerp
Faculty of Science

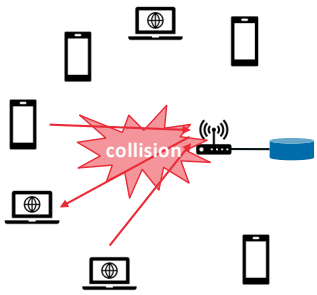
Section 6.3, 7.2.1, and 7.3.2

There are two types of network links: point-to-point links and broadcast links. A **point-to-point link** consists of a single sender at one end of the link and a single receiver at the other end of the link. Many link-layer protocols have been designed for point-to-point links; the point-to-point protocol (PPP) and high-level data link control (HDLC) are two such protocols. The second type of link, a **broadcast link**, can have multiple sending and receiving nodes all connected to the same, single, shared broadcast channel. The term *broadcast* is used here because when any one node transmits a frame, the channel broadcasts the frame and each of the other nodes receives a copy. Ethernet and wireless LANs are examples of broadcast link-layer technologies. For now, we'll take a step back from specific link-layer protocols and first examine a problem of central importance to the link layer: how to coordinate the access of multiple sending and receiving nodes to a shared broadcast channel—the **multiple access problem**. Broadcast channels are often used in LANs, networks that are geographically concentrated in a single building (or on a corporate or university campus). Thus, we'll look at how multiple access channels are used in LANs at the end.

The multiple access problem

- When many nodes transmit and receive data over a shared broadcast medium, a **multiple access protocol** is needed to avoid **collisions** (i.e., multiple transmissions at the same time)
- Many types of networks make use of a broadcast medium (e.g., cable access networks, Wi-Fi, satellite)

- Multiple access protocols can be classified in three categories:
  - Channel partitioning protocols
  - Random access protocols
  - Taking-turns protocols



Computer networks have protocols—so-called **multiple access protocols**—by which nodes regulate their transmission into the shared broadcast channel. Multiple access protocols are needed in a wide variety of network settings, including both wired and wireless access networks, and satellite networks.

Because all nodes are capable of transmitting frames, more than two nodes can transmit frames at the same time. When this happens, all of the nodes receive multiple frames at the same time; that is, the transmitted frames **collide** at all of the receivers. Typically, when there is a collision, none of the receiving nodes can make any sense of any of the frames that were transmitted; in a sense, the signals of the colliding frames become inextricably tangled together. Thus, all the frames involved in the collision are lost, and the broadcast channel is wasted during the collision interval. Clearly, if many nodes want to transmit frames frequently, many transmissions will result in collisions, and much of the bandwidth of the broadcast channel will be wasted.

In order to ensure that the broadcast channel performs useful work when multiple nodes are active, it is necessary to somehow coordinate the transmissions of the active nodes. This coordination job is the responsibility of the multiple access protocol. Over the years, dozens of multiple access protocols have been implemented in a variety of link-layer technologies. Nevertheless, we can classify just about any multiple access protocol as belonging to one of three categories: **channel partitioning protocols**, **random access protocols**, and **taking-turns protocols**.

Desirable characteristics of multiple access protocols

Given a broadcast channel of rate R bits per second (bps), a multiple access protocol should have the following desirable characteristics:

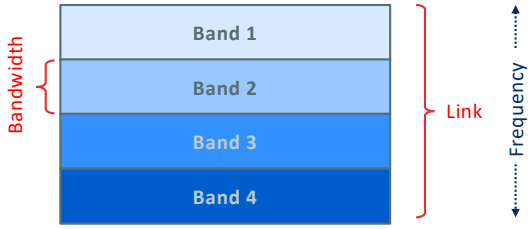
1. When only one node has data to send, that node has a throughput of *R* bps
2. When M nodes have data to send, each should have an average throughput of *R / M* bps
3. The protocol is decentralized (i.e., there is no master node that acts as a single point of failure)
4. The protocol is simple and inexpensive to implement

Ideally, a multiple access protocol for a broadcast channel of rate *R* bits per second should have the following desirable characteristics:

1. When only one node has data to send, that node has a throughput of *R* bps.
2. When *M* nodes have data to send, each of these nodes has a throughput of *R*/*M* bps. This need not necessarily imply that each of the *M* nodes always has an instantaneous rate of *R*/*M*, but rather that each node should have an average transmission rate of *R*/*M* over some suitably defined interval of time.
3. The protocol is decentralized; that is, there is no master node that represents a single point of failure for the network.
4. The protocol is simple, so that it is inexpensive to implement.

Channel partitioning protocols: FDMA and TDMA

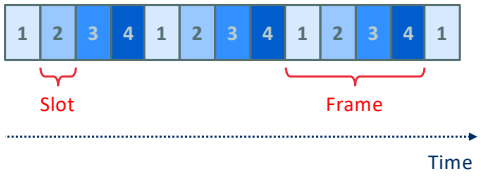
Frequency Division Multiple Access (FDMA)



Each frequency band is dedicated to a specific sender-receiver pair

Orthogonal FDMA (OFDMA) is used in 4G, 5G, and Wi-Fi 6

Time Division Multiple Access (TDMA)



All slots with the same label are dedicated to a specific sender-receiver pair

TDMA is used in 2G mobile networks and DECT

University of Antwerp
Faculty of Science

25

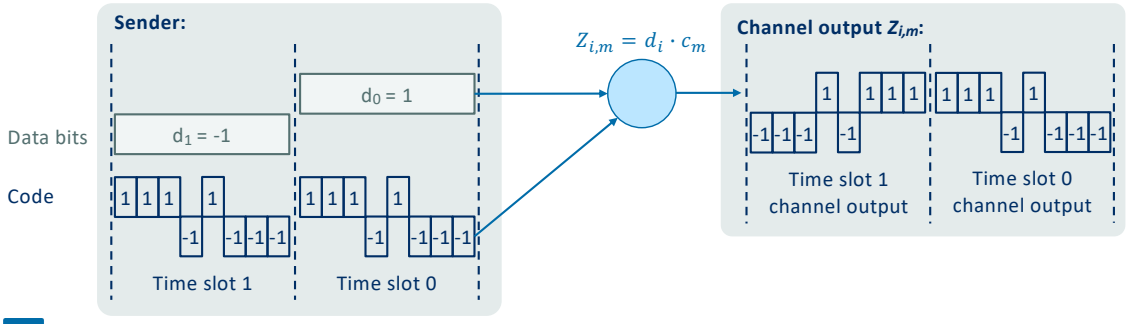
Recall that time-division multiplexing (TDM) and frequency-division multiplexing (FDM) are two techniques that can be used to partition a broadcast channel's bandwidth among all nodes sharing that channel. As an example, suppose the channel supports *N* nodes and that the transmission rate of the channel is *R* bps. TDM divides time into **time frames** and further divides each time frame into *N* **time slots**. Each time slot is then assigned to one of the *N* nodes. Whenever a node has a packet to send, it transmits the packet's bits during its assigned time slot in the revolving TDM frame. Typically, slot sizes are chosen so that a single packet can be transmitted during a slot time. TDM is appealing because it eliminates collisions and is perfectly fair: Each node gets a dedicated transmission rate of *R*/*N* bps during each frame time. However, it has two major drawbacks. First, a node is limited to an average rate of *R*/*N* bps even when it is the only node with packets to send. A second drawback is that a node must always wait for its turn in the transmission sequence—again, even when it is the only node with a frame to send.

While TDM shares the broadcast channel in time, FDM divides the *R* bps channel into different frequencies (each with a bandwidth of *R*/*N*) and assigns each frequency to one of the *N* nodes. FDM thus creates *N* smaller channels of *R*/*N* bps out of the single, larger *R* bps channel. FDM shares both the advantages and drawbacks of TDM. It avoids collisions and divides the bandwidth fairly among the *N* nodes. However, FDM also shares a principal disadvantage with TDM—a node is limited to a bandwidth of *R*/*N*, even when it is the only node with packets to send.

Channel partitioning protocols: CDMA

Code Division Multiple Access (CDMA)

- Each sender is assigned a unique M-bit code
- The code changes at a much faster rate than the sequence of data bits (i.e., the chipping rate)
- Each transmitted bit is encoded by multiplying it by the code
- Example (for convenience, a value of -1 is used for 0 bits):



University of Antwerp
Faculty of Science

26

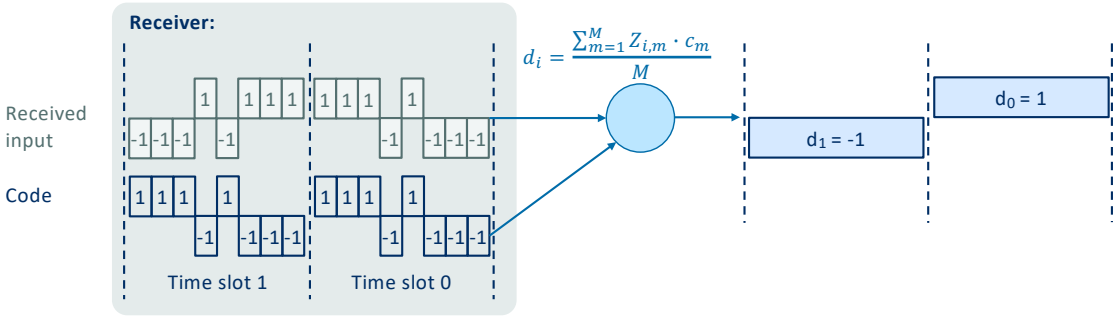
In a code division multiple access (CDMA) protocol, each bit being sent is encoded by multiplying the bit by a signal (the code) that changes at a much faster rate (known as the **chipping rate**) than the original sequence of data bits.

The figure shows a simple, idealized CDMA encoding/decoding scenario. Suppose that the rate at which original data bits reach the CDMA encoder defines the unit of time; that is, each original data bit to be transmitted requires a one-bit slot time. Let d_i be the value of the data bit for the *i*th bit slot. For mathematical convenience, we represent a data bit with a 0 value as -1. Each bit slot is further subdivided into *M* mini-slots; here, *M* = 8, although in practice *M* is much larger. The CDMA code used by the sender consists of a sequence of *M* values, c_m, *m* = [1, . . . , *M*], each taking a +1 or -1 value. In the example, the *M*-bit CDMA code being used by the sender is (1, 1, 1, -1, 1, -1, -1, -1).

To illustrate how CDMA works, let us focus on the *i*th data bit, d_i. For the *m*th mini-slot of the bit-transmission time of d_i, the output of the CDMA encoder, $Z_{i,m}$, is the value of d_i multiplied by the *m*th bit in the assigned CDMA code, c_m:

$$Z_{i,m} = d_i \cdot c_m$$

Channel partitioning protocols: CDMA receiver



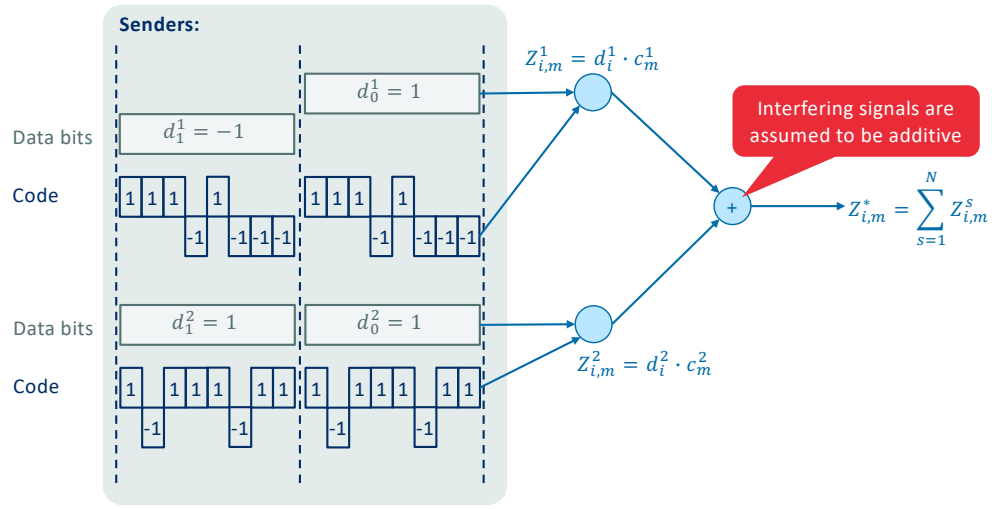
University of Antwerp
Faculty of Science

27

In a simple world, with no interfering senders, the receiver would receive the encoded bits, $Z_{i,m}$, and recover the original data bit, d_i, by computing:

$$d_i = \frac{\sum_{m=1}^{M} Z_{i,m} \cdot c_m}{M}$$

Channel partitioning protocols: CDMA with multiple transmitters



University of Antwerp
Faculty of Science

28

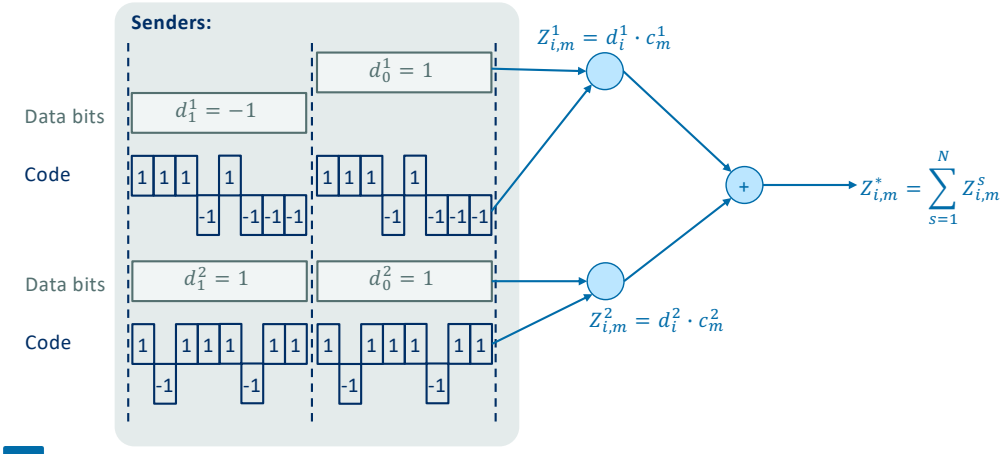
The world is far from ideal, however, and CDMA must work in the presence of interfering senders that are encoding and transmitting their data using a different assigned code. But how can a CDMA receiver recover a sender's original data bits when those data bits are being tangled with bits being transmitted by other senders? CDMA works under the assumption that the interfering transmitted bit signals are additive. This means, for example, that if three senders send a 1 value, and a fourth sender sends a -1 value during the same mini-slot, then the received signal at all receivers during that mini-slot is a 2 (since 1 + 1 + 1 - 1 = 2). In the presence of multiple senders, sender *s* computes its encoded transmissions, $Z^s_{i,m}$, in exactly the same manner as before. The value received at a receiver during the *m*th mini-slot of the *i*th bit slot, however, is now the *sum* of the transmitted bits from all *N* senders during that mini-slot:

$$Z^*_{i,m} = \sum_{s=1}^{N} Z^s_{i,m}$$

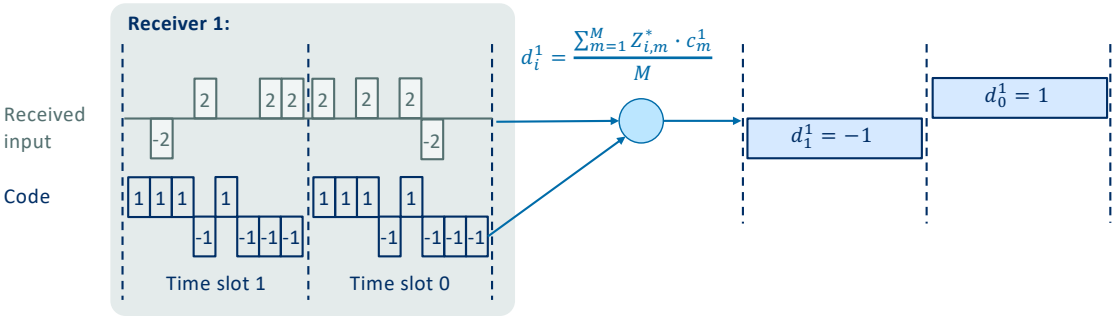
As shown in the figure, for a two-sender CDMA example. The *M*-bit CDMA code being used by the upper sender is (1, 1, 1, -1, 1, -1, -1, -1), while the CDMA code being used by the lower sender is (1, -1, 1, 1, 1, -1, 1, 1).

Exercise: CDMA with multiple transmitters

Exercise: Calculate the resulting channel output $Z^*_{i,m}$ for the given example with two transmitters, transmitting 01 and 11 and using codes 11101000 and 10111011 respectively.



Channel partitioning protocols: CDMA receiver with two senders



Assumptions

- CDMA codes must be carefully chosen (i.e., orthogonal codes, such as Walsh-Hadamard, Gold codes, ...)
- Received signal strength from senders is the same (usually not true in practice)
- Transmissions of different users are time-synchronized (not easy in practice)

Used in 3G mobile networks

Amazingly, if the senders' codes are chosen carefully, each receiver can recover the data sent by a given sender out of the aggregate signal simply by using the sender's code in exactly the same manner as before (for sender s):

$$d_i^s = \frac{\sum_{m=1}^{M} Z_{i,m}^* \cdot c_m^s}{M}$$

The figure illustrates a receiver recovering the original data bits from the upper sender. Note that the receiver is able to extract the data from sender 1 in spite of the interfering transmission from sender 2. Our discussion here of CDMA is necessarily brief; in practice a number of difficult issues must be addressed. First, in order for the CDMA receivers to be able to extract a particular sender's signal, the CDMA codes must be carefully chosen. Second, our discussion has assumed that the received signal strengths from various senders are the same; in reality, this can be difficult to achieve. There is a considerable body of literature addressing these and other issues related to CDMA

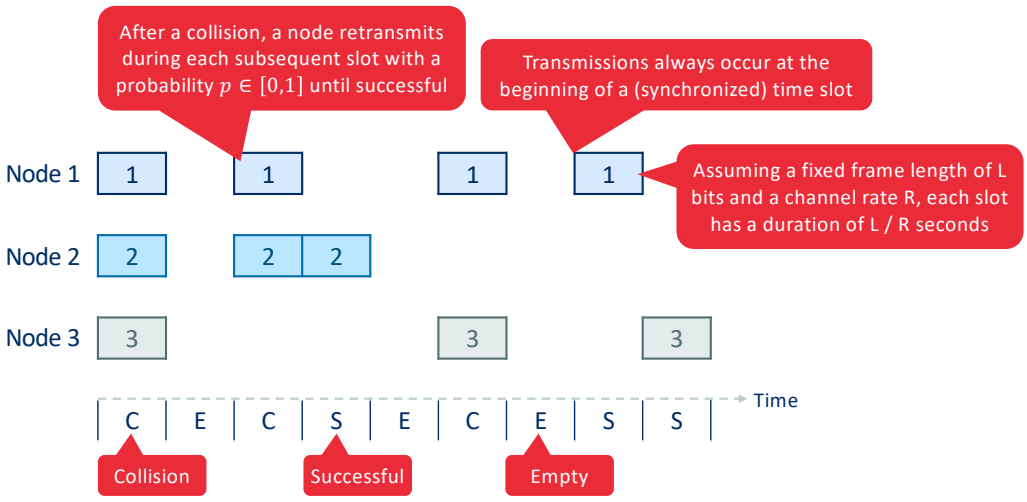
Random access protocols

- Each node transmits at the full rate of the channel
- When there is a collision, nodes repeatedly retransmit their frame
- Each node waits a random delay before retransmitting
- Collisions are avoided if the random delays are different enough

- Well known random access protocols:
  - (slotted) ALOHA
  - Carrier sense multiple access (CSMA), used by Ethernet and Wi-Fi

The second broad class of multiple access protocols are random access protocols. In a random access protocol, a transmitting node always transmits at the full rate of the channel, namely, *R* bps. When there is a collision, each node involved in the collision repeatedly retransmits its frame (that is, packet) until its frame gets through without a collision. But when a node experiences a collision, it doesn't necessarily retransmit the frame right away. *Instead it waits a random delay before retransmitting the frame*. Each node involved in a collision chooses independent random delays. Because the random delays are independently chosen, it is possible that one of the nodes will pick a delay that is sufficiently less than the delays of the other colliding nodes and will therefore be able to sneak its frame into the channel without a collision.

Random access protocols: Slotted ALOHA



University of Antwerp
Faculty of Science

33

Let's begin our study of random access protocols with one of the simplest random access protocols, the slotted ALOHA protocol. In our description of slotted ALOHA, we assume the following:

- All frames consist of exactly *L* bits.
- Time is divided into slots of size *L*/*R* seconds (that is, a slot equals the time to transmit one frame).
- Nodes start to transmit frames only at the beginnings of slots.
- The nodes are synchronized so that each node knows when the slots begin.
- If two or more frames collide in a slot, then all the nodes detect the collision event before the slot ends.

Let *p* be a probability, that is, a number between 0 and 1. The operation of slotted ALOHA in each node is simple:

- When the node has a fresh frame to send, it waits until the beginning of the next slot and transmits the entire frame in the slot.
- If there isn't a collision, the node has successfully transmitted its frame and thus need not consider retransmitting the frame. (The node can prepare a new frame for transmission, if it has one.)
- If there is a collision, the node detects the collision before the end of the slot. The node retransmits its frame in each subsequent slot with probability *p* until the frame is transmitted without a collision.

By retransmitting with probability *p*, we mean that the node effectively tosses a biased coin; the event heads corresponds to "retransmit," which occurs with probability *p*. The event tails corresponds to "skip the slot and toss the coin again in the next slot"; this occurs with probability (1 - *p*). All nodes involved in the collision toss their coins independently.

Random access protocols: Slotted ALOHA efficiency

- Assumption: Each node of the set of N nodes transmits during each slot with a probability *p*
- Probability that a **given node** transmits successfully (i.e., only that node attempts a transmission):
$$p \cdot (1-p)^{N-1}$$
- Probability that any node transmits successfully:
$$N \cdot p \cdot (1-p)^{N-1}$$
- The maximum efficiency of slotted ALOHA is achieved by finding p^*, such that:
$$p^* = \underset{p}{\operatorname{argmax}}(N \cdot p \cdot (1-p)^{N-1})$$
- For a large number of senders (i.e., the limit at $N \to \infty$), the maximum efficiency is $\frac{1}{e} \cong 0.37$

i.e., only 37% of the bandwidth is utilized for successful transmissions

Slotted ALOHA works well when there is only one active node, but how efficient is it when there are multiple active nodes? There are two possible efficiency concerns here. First, when there are multiple active nodes, a certain fraction of the slots will have collisions and will therefore be "wasted." The second concern is that another fraction of the slots will be *empty* because all active nodes refrain from transmitting as a result of the probabilistic transmission policy. The only "unwasted" slots will be those in which exactly one node transmits. A slot in which exactly one node transmits is said to be a **successful slot**. The **efficiency** of a slotted multiple access protocol is defined to be the long-run fraction of successful slots in the case when there are a large number of active nodes, each always having a large number of frames to send.

We now proceed to outline the derivation of the maximum efficiency of slotted ALOHA. To keep this derivation simple, let's modify the protocol a little and assume that each node attempts to transmit a frame in each slot with probability *p*. (That is, we assume that each node always has a frame to send and that the node transmits with probability *p* for a fresh frame as well as for a frame that has already suffered a collision.) Suppose there are *N* nodes. Then the probability that a given slot is a successful slot is the probability that one of the nodes transmits and that the remaining *N* - 1 nodes do not transmit. The probability that a given node transmits is *p;* the probability that the remaining nodes do not transmit is $(1 - p)^{N-1}$. Therefore, the probability a given node has a success is $p\ (1 - p)^{N-1}$. Because there are *N* nodes, the probability that any one of the *N* nodes has a success is $N\ p\ (1 - p)^{N-1}$.

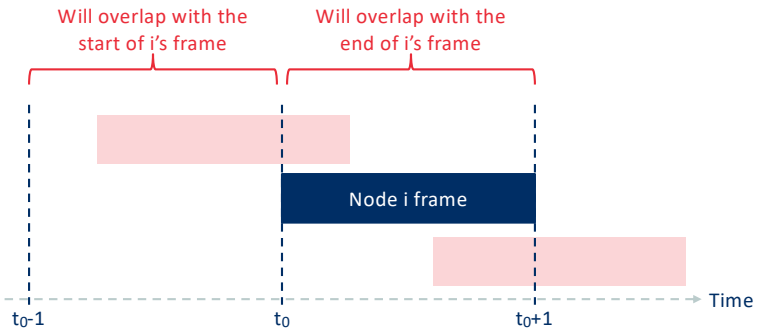
Thus, when there are *N* active nodes, the efficiency of slotted ALOHA is $N\ p\ (1 - p)^{N-1}$. To obtain the *maximum* efficiency for *N* active nodes, we have to find the p^* that maximizes this

expression. And to obtain the maximum efficiency for a large number of active nodes, we take the limit of $N\,p^*\,(1 - p^*)^{N-1}$ as N approaches infinity. After performing these calculations, we'll find that the maximum efficiency of the protocol is given by $1/e = 0.37$. That is, when a large number of nodes have many frames to transmit, then (at best) only 37 percent of the slots do useful work. Thus, the effective transmission rate of the channel is not R bps but only 0.37 R bps! A similar analysis also shows that 37 percent of the slots go empty and 26 percent of slots have collisions.

Complete proof can be found online:
http://web.cs.ucla.edu/~zyuan/teaching/spring19/cs118/aloha-proof.pdf

Random access protocols: Pure ALOHA

- Node immediately transmits a frame, instead of waiting for a start of the next time slot
- If a collision occurs, the node immediately retransmits with probability *p*
- Otherwise, the node waits for a single frame transmission time and retransmits again with probability *p*



Efficiency of pure ALOHA

- Probability that no other node transmits during [t_0-1, t_0]: $(1-p)^{N-1}$
- Probability that no other node transmits during [t_0, t_0+1]: $(1-p)^{N-1}$
- Successful transmission probability: $p \cdot (1-p)^{2(N-1)}$
- Maximum efficiency: $\frac{1}{2e} \cong 0.18$

University of Antwerp | Faculty of Science

35

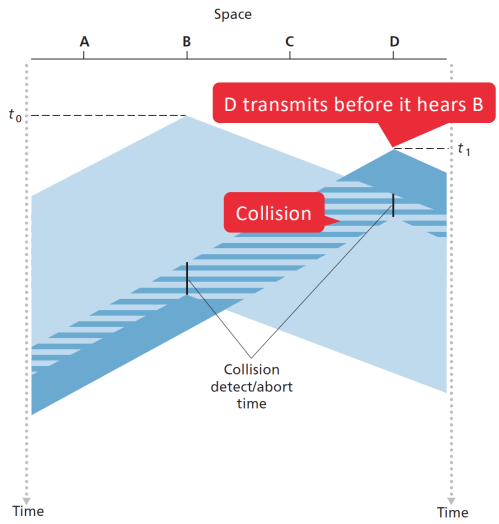
The slotted ALOHA protocol required that all nodes synchronize their transmissions to start at the beginning of a slot. In pure ALOHA, when a frame first arrives (that is, a network-layer datagram is passed down from the network layer at the sending node), the node immediately transmits the frame in its entirety into the broadcast channel. If a transmitted frame experiences a collision with one or more other transmissions, the node will then immediately (after completely transmitting its collided frame) retransmit the frame with probability *p*. Otherwise, the node waits for a frame transmission time. After this wait, it then transmits the frame with probability *p*, or waits (remaining idle) for another frame time with probability 1 – *p*.

To determine the maximum efficiency of pure ALOHA, we focus on an individual node. We'll make the same assumptions as in our slotted ALOHA analysis and take the frame transmission time to be the unit of time. At any given time, the probability that a node is transmitting a frame is *p*. Suppose this frame begins transmission at time *t*0. As shown in the figure, in order for this frame to be successfully transmitted, no other nodes can begin their transmission in the interval of time [t_0 - 1, t_0]. Such a transmission would overlap with the beginning of the transmission of node *i*'s frame. The probability that all other nodes do not begin a transmission in this interval is $(1 - p)^{N-1}$. Similarly, no other node can begin a transmission while node *i* is transmitting, as such a transmission would overlap with the latter part of node *i*'s transmission. The probability that all other nodes do not begin a transmission in this interval is also $(1 - p)^{N-1}$. Thus, the probability that a given node has a successful transmission is $p\ (1 - p)^{2(N-1)}$. By taking limits as in the slotted ALOHA case, we find that the maximum efficiency of the pure ALOHA protocol is only $1/(2e)$—exactly half that of slotted ALOHA. This then is the price to be paid for a fully decentralized ALOHA protocol.

Random access protocols: CSMA/CD

Carrier Sense Multiple Access with Collision Detection (CSMA/CD)

- Makes use of two important rules to avoid collisions
  - **Carrier sensing**: A node listens to the channel before transmitting, and only transmits if it detects no other ongoing transmissions
  - **Collision detection**: If another transmission is detected while transmitting, stop transmitting and wait a random back-off time before retransmitting
- Collision detection is needed, as even with carrier sensing nodes may still transmit simultaneously



University of Antwerp
Faculty of Science

36

In both slotted and pure ALOHA, a node's decision to transmit is made independently of the activity of the other nodes attached to the broadcast channel. In particular, a node neither pays attention to whether another node happens to be transmitting when it begins to transmit, nor stops transmitting if another node begins to interfere with its transmission. As humans, we have human protocols that allow us not only to behave with more civility, but also to decrease the amount of time spent "colliding" with each other in conversation and, consequently, to increase the amount of data we exchange in our conversations. Specifically, there are two important rules for polite human conversation:

- *Listen before speaking.* If someone else is speaking, wait until they are finished. In the networking world, this is called **carrier sensing**—a node listens to the channel before transmitting. If a frame from another node is currently being transmitted into the channel, a node then waits until it detects no transmissions for a short amount of time and then begins transmission.
- *If someone else begins talking at the same time, stop talking.* In the networking world, this is called **collision detection**—a transmitting node listens to the channel while it is transmitting. If it detects that another node is transmitting an interfering frame, it stops transmitting and waits a random amount of time before repeating the sense-and-transmit-when-idle cycle.

These two rules are embodied in the family of **carrier sense multiple access (CSMA)** and **CSMA with collision detection (CSMA/CD)** protocols.

The first question that you might ask about CSMA is why, if all nodes perform carrier sensing, do collisions occur in the first place? After all, a node will refrain from transmitting whenever

it senses that another node is transmitting. The answer to the question can best be illustrated using space-time diagrams. The figure shows a space-time diagram of four nodes (A, B, C, D) attached to a linear broadcast bus. The horizontal axis shows the position of each node in space; the vertical axis represents time.

At time t_0, node B senses the channel is idle, as no other nodes are currently transmitting. Node B thus begins transmitting, with its bits propagating in both directions along the broadcast medium. The downward propagation of B's bits with increasing time indicates that a nonzero amount of time is needed for B's bits actually to propagate (albeit at near the speed of light) along the broadcast medium. At time t_1 ($t_1 > t_0$), node D has a frame to send. Although node B is currently transmitting at time t_1, the bits being transmitted by B have yet to reach D, and thus D senses the channel idle at t_1. In accordance with the CSMA protocol, D thus begins transmitting its frame. A short time later, B's transmission begins to interfere with D's transmission at D. it is evident that the end-to-end **channel propagation delay** of a broadcast channel—the time it takes for a signal to propagate from one of the nodes to another—will play a crucial role in determining its performance. The longer this propagation delay, the larger the chance that a carrier-sensing node is not yet able to sense a transmission that has already begun at another node in the network. The two nodes each abort their transmission a short time after detecting a collision. Clearly, adding collision detection to a multiple access protocol will help protocol performance by not transmitting a useless, damaged (by interference with a frame from another node) frame in its entirety.

Random access protocols: CSMA/CD random back-off time

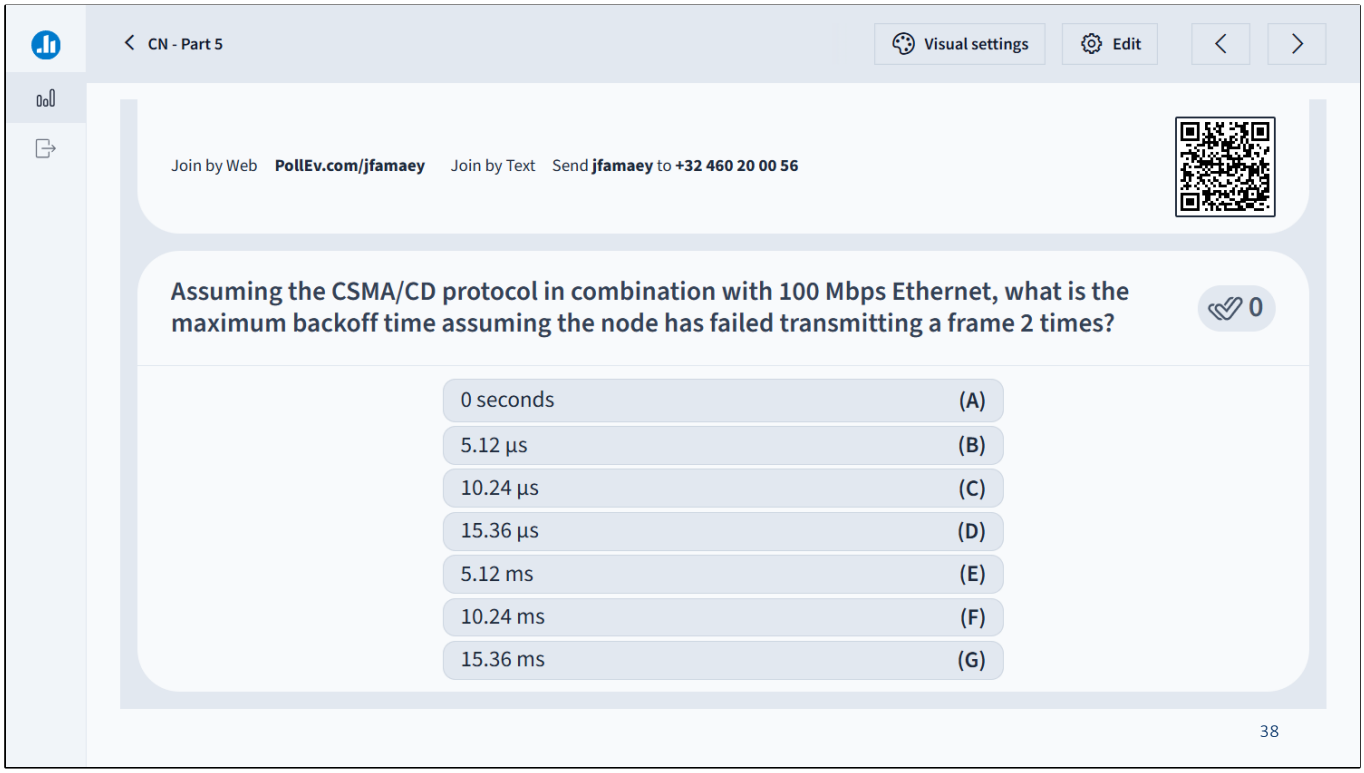
- If the back-off time is large and few nodes collide, then the channel will often remain idle
- If the back-off time is small and many nodes collide, then collisions will likely keep occurring
- Optimally, back-off should be long when many nodes collide, and small when few nodes collide

- **Answer: Binary exponential backoff** (used by Ethernet and DOCSIS protocols)
  - After *n* subsequent collisions, select back-off time of K timeslots uniformly at random from {0, 1, 2, ..., $2^n - 1$}
  - After a successful transmission, the value of n is set to 0 again
  - Specifically for Ethernet, the actual time equals K x 512 bit times (i.e., K times amount of time needed to transmit 512 bits), with a maximum value for n equal to 10

The need to wait a random (rather than fixed) amount of time is hopefully clear—if two nodes transmitted frames at the same time and then both waited the same fixed amount of time, they'd continue colliding forever. But what is a good interval of time from which to choose the random backoff time? If the interval is large and the number of colliding nodes is small, nodes are likely to wait a large amount of time (with the channel remaining idle) before repeating the sense-and-transmit-when-idle step. On the other hand, if the interval is small and the number of colliding nodes is large, it's likely that the chosen random values will be nearly the same, and transmitting nodes will again collide. What we'd like is an interval that is short when the number of colliding nodes is small, and long when the number of colliding nodes is large.

The **binary exponential backoff** algorithm, used in Ethernet as well as in DOCSIS cable network multiple access protocols, elegantly solves this problem. Specifically, when transmitting a frame that has already experienced *n* collisions, a node chooses the value of *K* at random from {0,1,2, ..., 2^n-1}. Thus, the more collisions experienced by a frame, the larger the interval from which *K* is chosen. For Ethernet, the actual amount of time a node waits is K x 512 bit times (i.e., *K* times the amount of time needed to send 512 bits into the Ethernet) and the maximum value that *n* can take is capped at 10.

We also note here that each time a node prepares a new frame for transmission, it runs the CSMA/CD algorithm, not taking into account any collisions that may have occurred in the recent past. So it is possible that a node with a new frame will immediately be able to sneak in a successful transmission while several other nodes are in the exponential backoff state.


CN - Part 5
Visual settings
Edit
Join by Web PollEv.com/jfamaey Join by Text Send jfamaey to +32 460 20 00 56
Assuming the CSMA/CD protocol in combination with 100 Mbps Ethernet, what is the maximum backoff time assuming the node has failed transmitting a frame 2 times?
0
0 seconds (A)
5.12 µs (B)
10.24 µs (C)
15.36 µs (D)
5.12 ms (E)
10.24 ms (F)
15.36 ms (G)
38

Random access protocols: CSMA/CD efficiency

- Assumption: A large number of nodes continuously transmitting frames
- Given a maximum propagation delay d_{prop} and a maximum frame transmission time d_{trans}, the efficiency of Ethernet is as follows:

$$\text{Efficiency} = \frac{1}{1 + 5 \cdot {}^{d_{prop}}\!/_{d_{trans}}}$$

- Examples (for nodes 10 meters away, d_{prop} = 0.043µs)
  - 10 Mbps Ethernet (d_{trans} = 1.21ms): Efficiency = 0.9998
  - 100 Mbps Ethernet (d_{trans} = 121.44µs) = 0.998
  - 1 Gbps Ethernet (d_{trans} = 12,14µs) = 0.98
- Examples (for nodes 2500 meters away, d_{prop} = 10.75µs)
  - 10 Mbps Ethernet (d_{trans} = 1.21ms): Efficiency = 0.96
  - 100 Mbps Ethernet (d_{trans} = 121.44µs) = 0.69
  - 1 Gbps Ethernet (d_{trans} = 12,14µs) = 0.18

University of Antwerp
Faculty of Science

40

When only one node has a frame to send, the node can transmit at the full channel rate (e.g., for Ethernet typical rates are 10 Mbps, 100 Mbps, or 1 Gbps). However, if many nodes have frames to transmit, the effective transmission rate of the channel can be much less. We define the **efficiency of CSMA/CD** to be the long-run fraction of time during which frames are being transmitted on the channel without collisions when there is a large number of active nodes, with each node having a large number of frames to send.

In order to present a closed-form approximation of the efficiency of Ethernet, let d_{prop} denote the maximum time it takes signal energy to propagate between any two adapters. Let d_{trans} be the time to transmit a maximum-size frame (approximately 1.2 msecs for a 10 Mbps Ethernet). A derivation of the efficiency of CSMA/CD is beyond the scope. Here we simply state the following approximation:

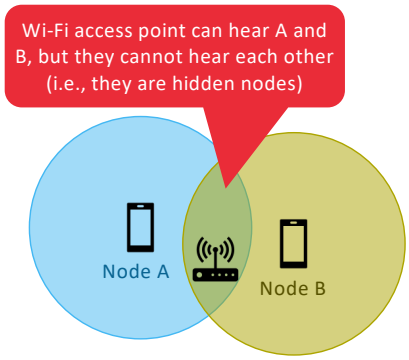
$$\text{Efficiency} = \frac{1}{1 + 5 \cdot {}^{d_{prop}}\!/_{d_{trans}}}$$

We see from this formula that as d_{prop} approaches 0, the efficiency approaches 1. This matches our intuition that if the propagation delay is zero, colliding nodes will abort immediately without wasting the channel. Also, as d_{trans} becomes very large, efficiency approaches 1. This is also intuitive because when a frame grabs the channel, it will hold on to the channel for a very long time; thus, the channel will be doing productive work most of the time.

Random access protocols: CSMA/CA

Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)

- Used by Wi-Fi, due to difficulty of performing **collision detection** in wireless networks:
  - In Wi-Fi transmission power is generally much higher than reception power, making it hard (and costly!) to build hardware that can listen while transmitting
  - Due to the hidden terminal problem, not all transmissions can be heard by all nodes in the network
- Wi-Fi uses **collision avoidance** instead
  - If a node has a frame to transmit and the channel is sensed busy, it immediately starts a random backoff counter (using binary exponential backoff)
  - While the channel is busy, the timer is frozen
  - When the counter reaches zero, the frame is transmitted fully and the node waits for an acknowledgment
  - If no acknowledgment is received within a timeout, the maximum backoff is increased and a new random backoff counter is started



University of Antwerp
Faculty of Science

41

Recall that with Ethernet's collision-detection algorithm, an Ethernet station listens to the channel as it transmits. If, while transmitting, it detects that another station is also transmitting, it aborts its transmission and tries to transmit again after waiting a small, random amount of time. Unlike the 802.3 Ethernet protocol, the 802.11 Wi-Fi MAC protocol does *not* implement collision detection. There are two important reasons for this:

- The ability to detect collisions requires the ability to send (the station's own signal) and receive (to determine whether another station is also transmitting) at the same time. Because the strength of the received signal is typically very small compared to the strength of the transmitted signal at the 802.11 adapter, it is costly to build hardware that can detect a collision.
- More importantly, even if the adapter could transmit and listen at the same time (and presumably abort transmission when it senses a busy channel), the adapter would still not be able to detect all collisions, due to the hidden terminal problem and fading.

Because 802.11 wireless LANs do not use collision detection, once a station begins to transmit a frame, *it transmits the frame in its entirety*; that is, once a station gets started, there is no turning back. As one might expect, transmitting entire frames (particularly long frames) when collisions are prevalent can significantly degrade a multiple access protocol's performance. In order to reduce the likelihood of collisions, 802.11 employs several collision-avoidance techniques.

Suppose that a station (wireless device or an AP) has a frame to transmit.

1. If initially the station senses the channel idle, it transmits its frame after a short period of time known as the **Distributed Inter-frame Space (DIFS)**

2. Otherwise, the station chooses a random backoff value using binary exponential backoff and counts down this value after DIFS when the channel is sensed idle. While the channel is sensed busy, the counter value remains frozen.
3. 3When the counter reaches zero (note that this can only occur while the channel is sensed idle), the station transmits the entire frame and then waits for an acknowledgment.
4. If an acknowledgment is received, the transmitting station knows that its frame has been correctly received at the destination station. If the station has another frame to send, it begins the CSMA/CA protocol at step 2. If the acknowledgment isn't received, the transmitting station reenters the backoff phase in step 2, with the random value chosen from a larger interval.

Recall that under Ethernet's CSMA/CD, multiple access protocol, a station begins transmitting as soon as the channel is sensed idle. With CSMA/CA, however, the station refrains from transmitting while counting down, even when it senses the channel to be idle.

Taking-turns protocols

- Random access protocols (ALOHA, CSMA) do not possess the desirable property that when M nodes are active, their average throughput should be close to R / M bps, which taking-turns protocols do achieve
- **Polling protocol**: A master node polls each node in a round-robin fashion
  - Example: Bluetooth protocol
  - Advantages: Eliminates collisions and empty slots that occur in random access protocols
  - Disadvantages: Polling causes delays, master node results in a single point-of-failure
- **Token-passing protocol**: Only the node holding the token can transmit, and passes the token along after sending a maximum number of frames (or having no more frames left to send)
  - Examples: fibre distributed data interface (FDDI), IEEE 802.5 token ring protocol
  - Advantages: Eliminates collisions and empty slots
  - Disadvantages: Complex failure cases (e.g., a single node failure can crash the channel, recovering from lost tokens, etc.)

Recall that two desirable properties of a multiple access protocol are (1) when only one node is active, the active node has a throughput of *R* bps, and (2) when *M* nodes are active, then each active node has a throughput of nearly *R*/*M* bps. The ALOHA and CSMA protocols have this first property but not the second. This has motivated researchers to create another class of protocols—the **taking-turns protocols**. As with random access protocols, there are dozens of taking-turns protocols, and each one of these protocols has many variations. We'll discuss two of the more important protocols here.

The first one is the **polling protocol**. The polling protocol requires one of the nodes to be designated as a master node. The master node **polls** each of the nodes in a round-robin fashion. In particular, the master node first sends a message to node 1, saying that it (node 1) can transmit up to some maximum number of frames. After node 1 transmits some frames, the master node tells node 2 it (node 2) can transmit up to the maximum number of frames. (The master node can determine when a node has finished sending its frames by observing the lack of a signal on the channel.) The procedure continues in this manner, with the master node polling each of the nodes in a cyclic manner.

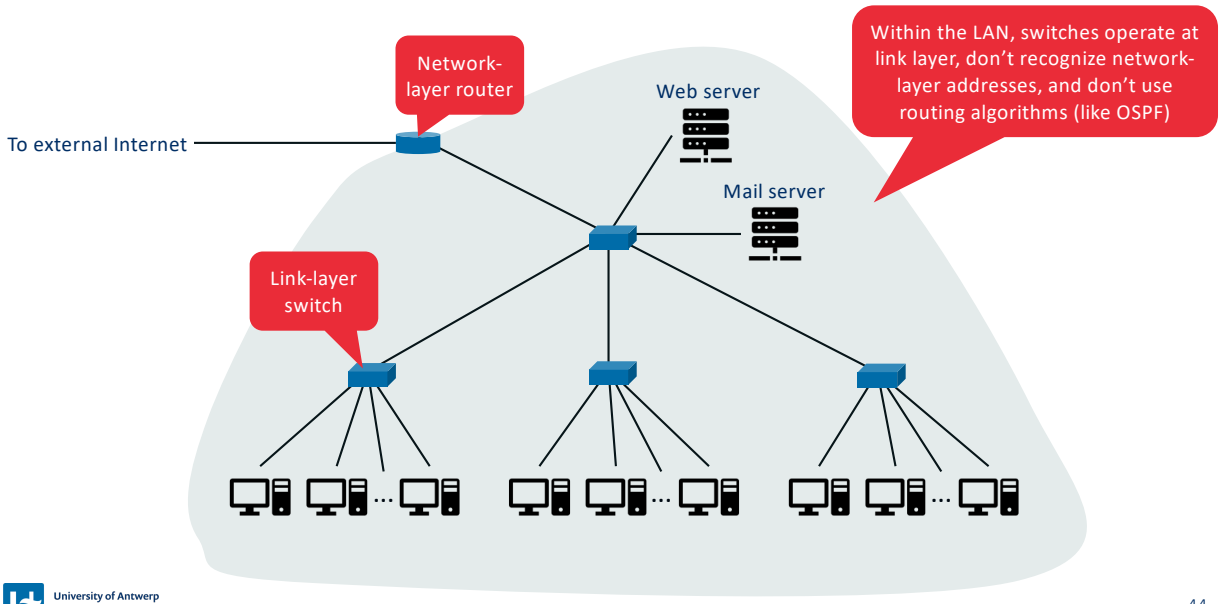
The polling protocol eliminates the collisions and empty slots that plague random access protocols. This allows polling to achieve a much higher efficiency. But it also has a few drawbacks. The first drawback is that the protocol introduces a polling delay—the amount of time required to notify a node that it can transmit. If, for example, only one node is active, then the node will transmit at a rate less than *R* bps, as the master node must poll each of the inactive nodes in turn each time the active node has sent its maximum number of frames. The second drawback, which is potentially more serious, is that if the master node fails, the

entire channel becomes inoperative. The Bluetooth protocol, is an example of a polling protocol.

The second taking-turns protocol is the **token-passing protocol**. In this protocol there is no master node. A small, special-purpose frame known as a **token** is exchanged among the nodes in some fixed order. For example, node 1 might always send the token to node 2, node 2 might always send the token to node 3, and node *N* might always send the token to node 1. When a node receives a token, it holds onto the token only if it has some frames to transmit; otherwise, it immediately forwards the token to the next node. If a node does have frames to transmit when it receives the token, it sends up to a maximum number of frames and then forwards the token to the next node. Token passing is decentralized and highly efficient. But it has its problems as well. For example, the failure of one node can crash the entire channel. Or if a node accidentally neglects to release the token, then some recovery procedure must be invoked to get the token back in circulation. Over the years many token-passing protocols have been developed, including the fiber distributed data interface (FDDI) protocol and the IEEE 802.5 token ring protocol, and each one had to address these as well as other sticky issues.

Local Area Networks (LANs)

Switched local area networks (LANs)



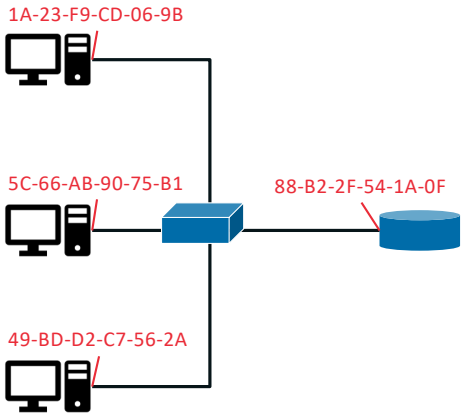
University of Antwerp
Faculty of Science

44

The figure shows a switched local network connecting three departments, two servers and a router with four switches. Because these switches operate at the link layer, they switch link-layer frames (rather than network-layer datagrams), don't recognize network-layer addresses, and don't use routing algorithms like OSPF to determine paths through the network of layer-2 switches. Instead of using IP addresses, we will soon see that they use link-layer addresses to forward link-layer frames through the network of switches.

Link-layer addressing

- Addresses at the link layer are called **MAC addresses**
- Each network adapter of a host and router has a MAC address
- Switches do not have MAC addresses (nor IP addresses)!
- Most LAN technologies (e.g., Ethernet, Wi-Fi) use 6-byte (48-bit) MAC address
- Originally MAC addresses were designed to be permanent, but now they can be changed via software
- Each vendor of network adapters is assigned a unique 24-bit MAC address prefix by IEEE, to ensure **all MAC addresses are unique**

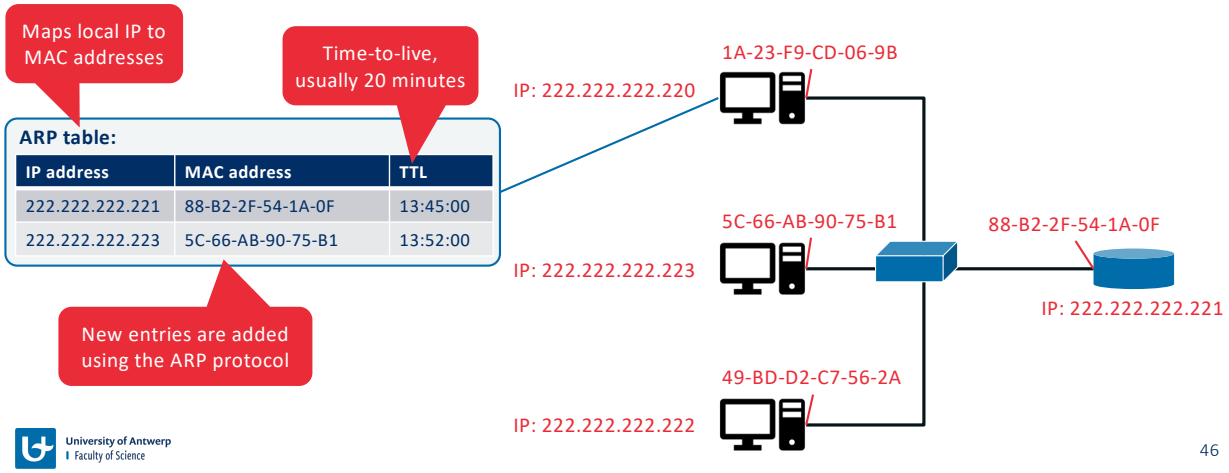


It is not hosts and routers that have link-layer addresses but rather their adapters (that is, network interfaces) that have link-layer addresses. A host or router with multiple network interfaces will thus have multiple link-layer addresses associated with it, just as it would also have multiple IP addresses associated with it. It's important to note, however, that link-layer switches do not have link-layer addresses associated with their interfaces that connect to hosts and routers. This is because the job of the link-layer switch is to carry datagrams between hosts and routers; a switch does this job transparently, that is, without the host or router having to explicitly address the frame to the intervening switch. A link-layer address is variously called a **LAN address**, a **physical address**, or a **MAC address**. Because MAC address seems to be the most popular term, we'll henceforth refer to link-layer addresses as MAC addresses. For most LANs (including Ethernet and 802.11 wireless LANs), the MAC address is 6 bytes long, giving 248 possible MAC addresses. Although MAC addresses were designed to be permanent, it is now possible to change an adapter's MAC address via software.

One interesting property of MAC addresses is that no two adapters have the same address. This might seem surprising given that adapters are manufactured in many countries by many companies. How does a company manufacturing adapters in Taiwan make sure that it is using different addresses from a company manufacturing adapters in Belgium? The answer is that the IEEE manages the MAC address space. In particular, when a company wants to manufacture adapters, it purchases a chunk of the address space consisting of 224 addresses for a nominal fee. IEEE allocates the chunk of 224 addresses by fixing the first 24 bits of a MAC address and letting the company create unique combinations of the last 24 bits for each adapter.

Address Resolution Protocol (ARP) – RFC 826

- Goal: Translate between network-layer address (e.g., IP) and link-layer addresses (i.e., MAC)
- ARP can provide the MAC address associated with an IP address **within the same LAN**

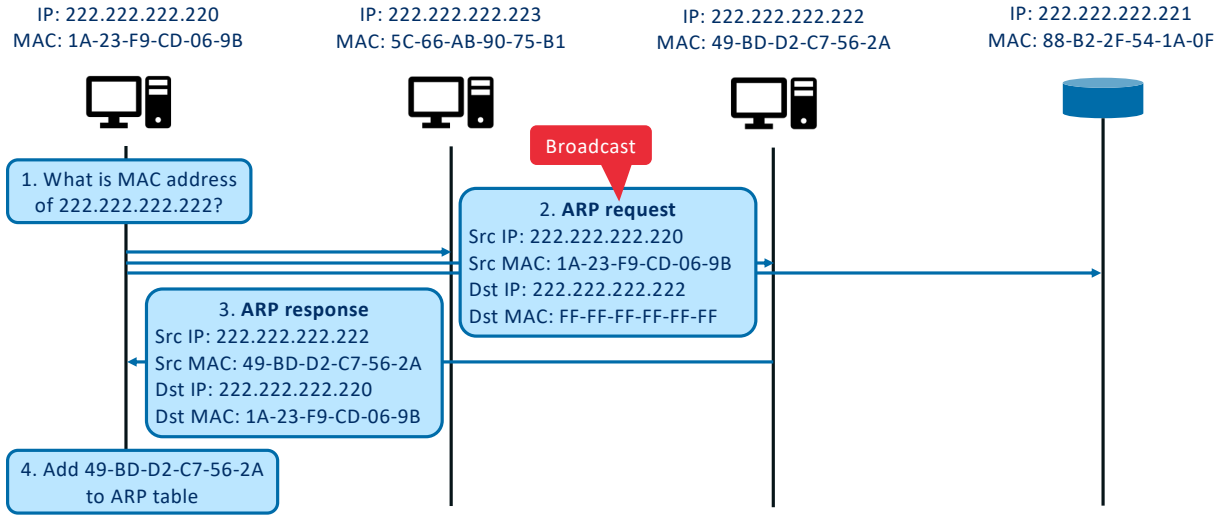


Because there are both network-layer addresses (for example, Internet IP addresses) and link-layer addresses (that is, MAC addresses), there is a need to translate between them. For the Internet, this is the job of the **Address Resolution Protocol (ARP)** [RFC 826].

To understand the need for a protocol such as ARP, consider the network shown in the figure. In this simple example, each host and router has a single IP address and single MAC address. As usual, IP addresses are shown in dotted-decimal notation and MAC addresses are shown in hexadecimal notation. Now suppose that the host with IP address 222.222.222.220 wants to send an IP datagram to host 222.222.222.222. How does the sending host determine the MAC address for the destination host with IP address 222.222.222.222? As you might have guessed, it uses ARP. An ARP module in the sending host takes any IP address on the same LAN as input, and returns the corresponding MAC address. In the example at hand, sending host 222.222.222.220 provides its ARP module the IP address 222.222.222.222, and the ARP module returns the corresponding MAC address 49-BD-D2-C7-56-2A.

Each host and router has an **ARP table** in its memory, which contains mappings of IP addresses to MAC addresses. The figure shows what an ARP table in host 222.222.222.220 might look like. The ARP table also contains a time-to-live (TTL) value, which indicates when each mapping will be deleted from the table. Note that a table does not necessarily contain an entry for every host and router on the subnet; some may have never been entered into the table, and others may have expired. A typical expiration time for an entry is 20 minutes from when an entry is placed in an ARP table.

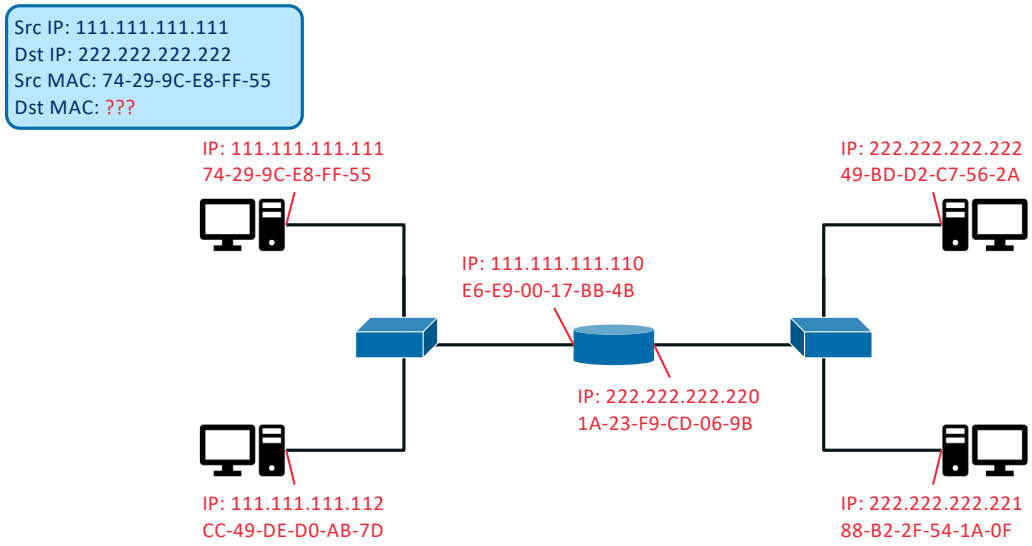
Address resolving using the ARP protocol within a subnet



Now suppose that host 222.222.222.220 wants to send a datagram that is IP-addressed to another host or router on that subnet. The sending host needs to obtain the MAC address of the destination given the IP address. This task is easy if the sender's ARP table has an entry for the destination node. But what if the ARP table doesn't currently have an entry for the destination? In particular, suppose 222.222.222.220 wants to send a datagram to 222.222.222.222. In this case, the sender uses the ARP protocol to resolve the address. First, the sender constructs a special packet called an **ARP packet**. An ARP packet has several fields, including the sending and receiving IP and MAC addresses. Both ARP query and response packets have the same format. The purpose of the ARP query packet is to query all the other hosts and routers on the subnet to determine the MAC address corresponding to the IP address that is being resolved.

222.222.222.220 passes an ARP query packet to the adapter along with an indication that the adapter should send the packet to the MAC broadcast address, namely, FF-FF-FF-FF-FF-FF. The adapter encapsulates the ARP packet in a link-layer frame, uses the broadcast address for the frame's destination address, and transmits the frame into the subnet. The frame containing the ARP query is received by all the other adapters on the subnet, and (because of the broadcast address) each adapter passes the ARP packet within the frame up to its ARP module. Each of these ARP modules checks to see if its IP address matches the destination IP address in the ARP packet. The one with a match sends back to the querying host a response ARP packet with the desired mapping. The querying host 222.222.222.220 can then update its ARP table and send its IP datagram, encapsulated in a link-layer frame whose destination MAC is that of the host or router responding to the earlier ARP query.

Sending datagrams across subnets

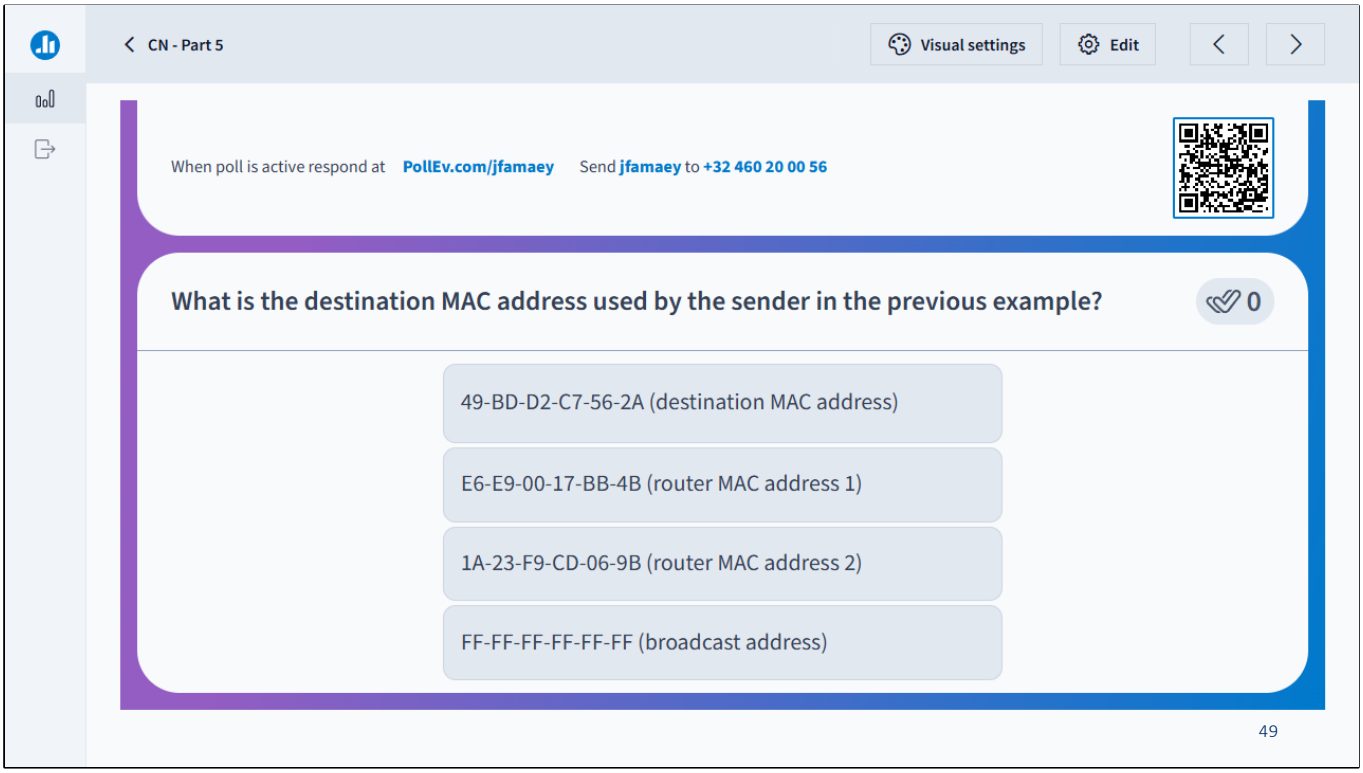


University of Antwerp
Faculty of Science

48

Let's look at the more complicated situation when a host on a subnet wants to send a network-layer datagram to a host *off the subnet* (that is, across a router onto another subnet).Subnet 1 has the network address 111.111.111/24 and that Subnet 2 has the network address 222.222.222/24. Thus, all of the interfaces connected to Subnet 1 have addresses of the form 111.111.111.xxx and all of the interfaces connected to Subnet 2 have addresses of the form 222.222.222.xxx.

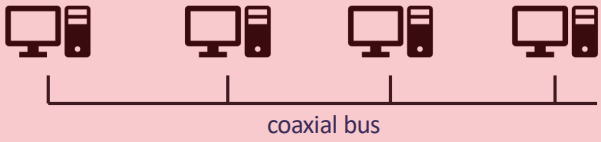
Now let's examine how a host on Subnet 1 would send a datagram to a host on Subnet 2. Specifically, suppose that host 111.111.111.111 wants to send an IP datagram to a host 222.222.222.222. The sending host passes the datagram to its adapter, as usual. But the sending host must also indicate to its adapter an appropriate destination MAC address. What MAC address should the adapter use?

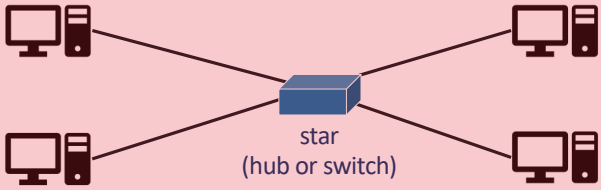

CN - Part 5
Visual settings
Edit
When poll is active respond at PollEv.com/jfamaey Send jfamaey to +32 460 20 00 56
What is the destination MAC address used by the sender in the previous example?
0
49-BD-D2-C7-56-2A (destination MAC address)
E6-E9-00-17-BB-4B (router MAC address 1)
1A-23-F9-CD-06-9B (router MAC address 2)
FF-FF-FF-FF-FF-FF (broadcast address)
49

NIET TE KENNEN VOOR EXAMEN

The history of Ethernet

- **1970s: Original Ethernet**
  - Invented by Bob Metcalfe and David Boggs
  - Uses coaxial bus to interconnect nodes in broadcast LAN
  - CSMA/CD needed to avoid collisions
- **1990s: Hub-based star topologies**
  - Hubs are physical layer devices that forward all incoming bits on all other interfaces
  - Also acts as a broadcast LAN (i.e., CSMA/CD required)
- **2000s: Switch-based star topologies**
  - Switches operate at the link layer
  - Collision-free store-and-forward packet switching
- Ethernet is **connectionless and unreliable**





University of Antwerp
Faculty of Science

51

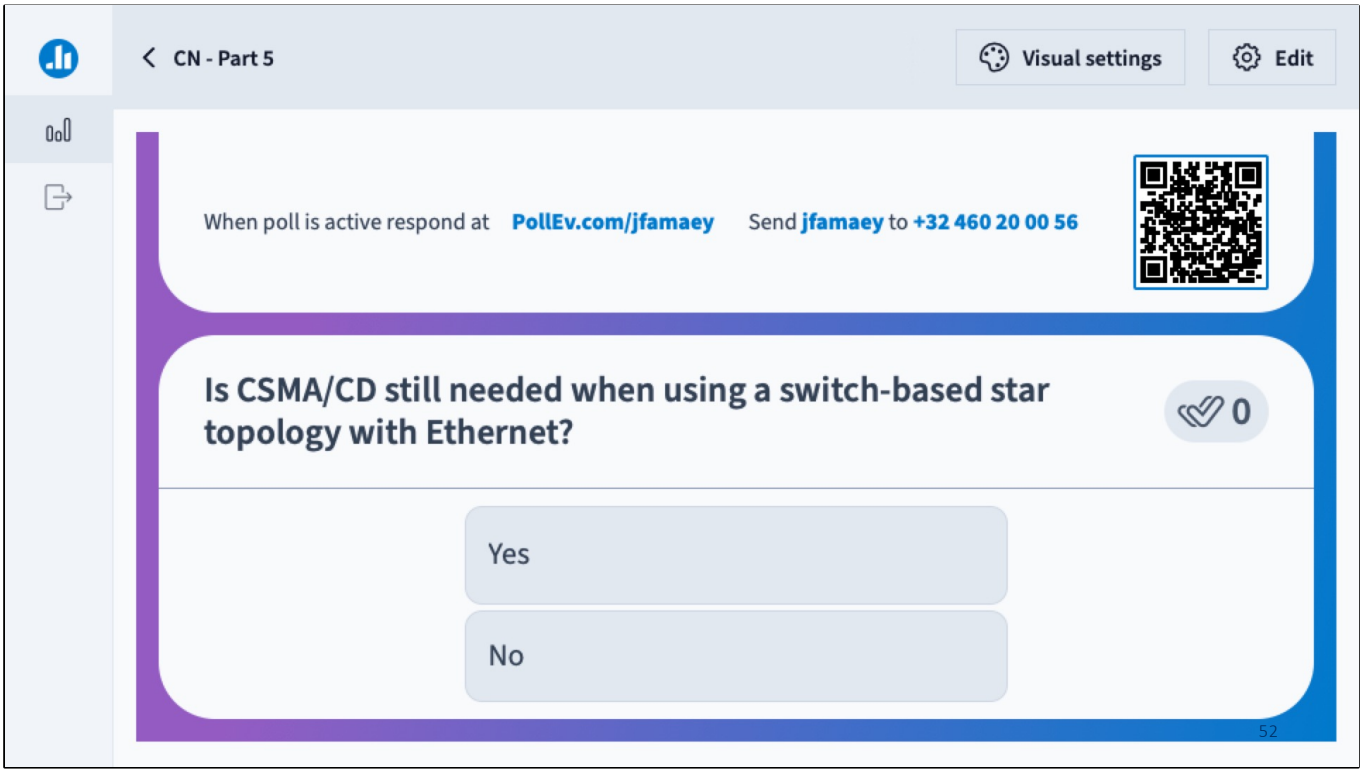
Today, Ethernet is by far the most prevalent wired LAN technology, and it is likely to remain so for the foreseeable future. One might say that Ethernet has been to local area networking what the Internet has been to global networking. The original Ethernet LAN was invented in the mid-1970s by Bob Metcalfe and David Boggs. The original Ethernet LAN used a coaxial bus to interconnect the nodes. Bus topologies for Ethernet actually persisted throughout the 1980s and into the mid-1990s. Ethernet with a bus topology is a broadcast LAN—all transmitted frames travel to and are processed by *all* adapters connected to the bus. Recall that we covered Ethernet's CSMA/CD multiple access protocol with binary exponential backoff.

By the late 1990s, most companies and universities had replaced their LANs with Ethernet installations using a hub-based star topology. In such an installation the hosts (and routers) are directly connected to a hub with twisted-pair copper wire. A **hub** is a physical-layer device that acts on individual bits rather than frames. When a bit, representing a zero or a one, arrives from one interface, the hub simply re-creates the bit, boosts its energy strength, and transmits the bit onto all the other interfaces. Thus, Ethernet with a hub-based star topology is also a broadcast LAN—whenever a hub receives a bit from one of its interfaces, it sends a copy out on all of its other interfaces. In particular, if a hub receives frames from two different interfaces at the same time, a collision occurs and the nodes that created the frames must retransmit.

In the early 2000s, Ethernet experienced yet another major evolutionary change. Ethernet installations continued to use a star topology, but the hub at the center was replaced with a **switch**. A switch is not only "collision-less" but is also a bona-fide store-and-forward packet

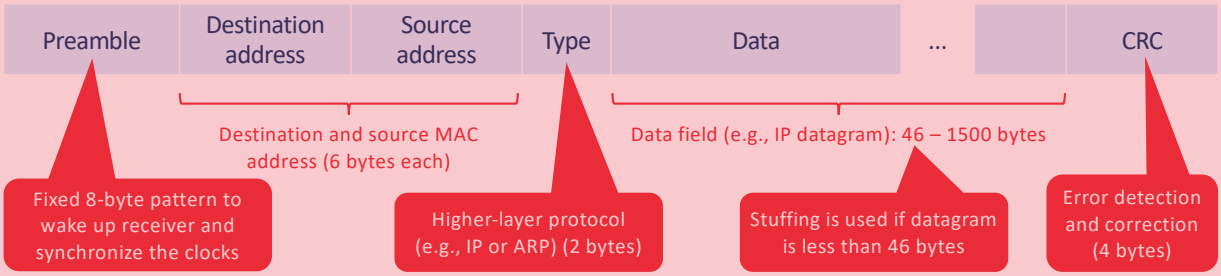
switch; but unlike routers, which operate up through layer 3, a switch operates only up through layer 2.

All of the Ethernet technologies provide connectionless service to the network layer. That is, when an adapter A wants to send a datagram to an adapter B, adapter A encapsulates the datagram in an Ethernet frame and sends the frame into the LAN, without first handshaking with adapter B. This layer-2 connectionless service is analogous to IP's layer-3 datagram service and UDP's layer-4 connectionless service. Ethernet technologies provide an unreliable service to the network layer. Specifically, when adapter B receives a frame from adapter A, it runs the frame through a CRC check, but neither sends an acknowledgment when a frame passes the CRC check nor sends a negative acknowledgment when a frame fails the CRC check. When a frame fails the CRC check, adapter B simply discards the frame. Thus, adapter A has no idea whether its transmitted frame reached adapter B and passed the CRC check. This lack of reliable transport (at the link layer) helps to make Ethernet simple and cheap. But it also means that the stream of datagrams passed to the network layer can have gaps.


CN - Part 5
Visual settings
Edit
When poll is active respond at PollEv.com/jfamaey
Send jfamaey to +32 460 20 00 56
Is CSMA/CD still needed when using a switch-based star topology with Ethernet?
0
Yes
No
52

NIET TE KENNEN VOOR EXAMEN

Ethernet frame structure



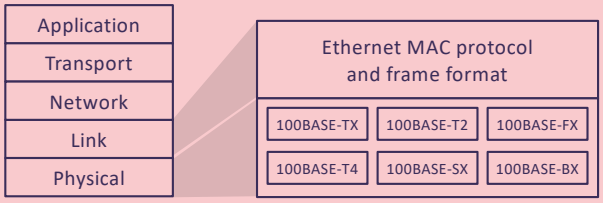
- ***Data field (46 to 1,500 bytes).*** This field carries the IP datagram. The maximum transmission unit (MTU) of Ethernet is 1,500 bytes. This means that if the IP datagram exceeds 1,500 bytes, then the host has to fragment the datagram. The minimum size of the data field is 46 bytes. This means that if the IP datagram is less than 46 bytes, the data field has to be "stuffed" to fill it out to 46 bytes. When stuffing is used, the data passed to the network layer contains the stuffing as well as an IP datagram. The network layer uses the length field in the IP datagram header to remove the stuffing.
- ***Destination address (6 bytes).*** This field contains the MAC address of the destination adapter. When an adapter receives an Ethernet frame whose destination address equals its own address or the MAC broadcast address, it passes the contents of the frame's data field to the network layer; if it receives a frame with any other MAC address, it discards the frame.
- ***Source address (6 bytes).*** This field contains the MAC address of the adapter that transmits the frame onto the LAN.
- ***Type field (2 bytes).*** The type field permits Ethernet to multiplex network-layer protocols. To understand this, we need to keep in mind that hosts can use other network-layer protocols besides IP. In fact, a given host may support multiple network-layer protocols using different protocols for different applications. IP and other network-layer protocols (for example, Novell IPX or AppleTalk) each have their own, standardized type number. Furthermore, the ARP protocol (discussed in the previous section) has its own type number, and if the arriving frame contains an ARP packet (i.e., has a type field of 0806 hexadecimal), the ARP packet will be demultiplexed up to the ARP protocol.
- ***Cyclic redundancy check (CRC) (4 bytes).*** The purpose of the CRC field is to allow the receiving adapter to detect bit errors in the frame.

- ***Preamble (8 bytes).*** The Ethernet frame begins with an 8-byte preamble field. Each of the first 7 bytes of the preamble has a value of 10101010; the last byte is 10101011. The first 7 bytes of the preamble serve to “wake up” the receiving adapters and to synchronize their clocks to that of the sender’s clock.

NIET TE KENNEN VOOR EXAMEN

Ethernet technologies

- Ethernet is both a physical and link layer technology
- Many different flavours, identified by acronym consisting of 3 parts (e.g., 100BASE-TX, 10GBASE-SR):
  - Part 1: Data rate (e.g., 10, 100, 1000, 10G, 40G for 10Mbps, 100Mbps, 1Gbps, 10Gbps, 40Gbps data rates)
  - Part 2: BASE, refers to baseband Ethernet (i.e., the physical medium only carries Ethernet traffic)
  - Part 3: Physical media used
    - T for twisted-pair copper
    - F for optical cable
    - S for multi-mode short optical cable
    - X/R for block coding type used
    - ...



University of Antwerp
Faculty of Science

55

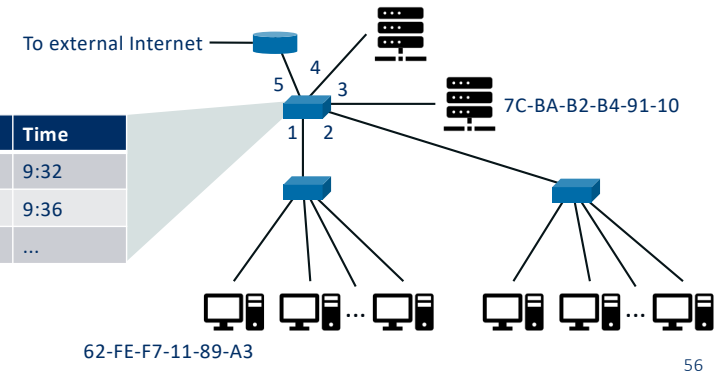
Ethernet comes in *many* different flavors, with somewhat bewildering acronyms such as 10BASE-T, 10BASE-2, 100BASE-T, 1000BASE-LX, 10GBASE-T and 40GBASE-T. These and many other Ethernet technologies have been standardized over the years by the IEEE 802.3 CSMA/CD (Ethernet) working group. While these acronyms may appear bewildering, there is actually considerable order here. The first part of the acronym refers to the speed of the standard: 10, 100, 1000, or 10G, for 10 Megabit (per second), 100 Megabit, Gigabit, 10 Gigabit and 40 Gigibit Ethernet, respectively. “BASE” refers to baseband Ethernet, meaning that the physical media only carries Ethernet traffic; almost all of the 802.3 standards are for baseband Ethernet. The final part of the acronym refers to the physical media itself; Ethernet is both a link-layer *and* a physical-layer specification and is carried over a variety of physical media including coaxial cable, copper wire, and fiber. Generally, a “T” refers to twisted-pair copper wires.

Link-layer switches: Forwarding and filtering

- **Filtering**: Determine whether a frame should be forwarded to some interface or should be dropped
- **Forwarding**: Determine the interfaces to which a frame should be directed (based on switch table)
  - Destination address not found: broadcast frame on all interfaces (except incoming interface)
  - Destination address is linked to incoming interface: filter (discard) frame
  - Destination address is linked to other interface: forward frame to that interface

Switch table:

Address	Interface	Time
62-FE-F7-11-89-A3	1	9:32
7C-BA-B2-B4-91-10	3	9:36
...	...	...



University of Antwerp
Faculty of Science

56

The role of the switch is to receive incoming link-layer frames and forward them onto outgoing links. We'll see that the switch itself is **transparent** to the hosts and routers in the subnet; that is, a host/router addresses a frame to another host/router (rather than addressing the frame to the switch) and happily sends the frame into the LAN, unaware that a switch will be receiving the frame and forwarding it. The rate at which frames arrive to any one of the switch's output interfaces may temporarily exceed the link capacity of that interface. To accommodate this problem, switch output interfaces have buffers, in much the same way that router output interfaces have buffers for datagrams.

Filtering is the switch function that determines whether a frame should be forwarded to some interface or should just be dropped. **Forwarding** is the switch function that determines the interfaces to which a frame should be directed, and then moves the frame to those interfaces. Switch filtering and forwarding are done with a **switch table**. The switch table contains entries for some, but not necessarily all, of the hosts and routers on a LAN. An entry in the switch table contains (1) a MAC address, (2) the switch interface that leads toward that MAC address, and (3) the time at which the entry was placed in the table.

To understand how switch filtering and forwarding work, suppose a frame with destination address DD-DD-DD-DD-DD-DD arrives at the switch on interface *x*. The switch indexes its table with the MAC address DD-DD-DD-DD-DD-DD. There are three possible cases:

- There is no entry in the table for DD-DD-DD-DD-DD-DD. In this case, the switch forwards copies of the frame to the output buffers preceding *all* interfaces except for interface *x*. In other words, if there is no entry for the destination address, the switch broadcasts the frame.

- There is an entry in the table, associating DD-DD-DD-DD-DD-DD with interface *x*. In this case, the frame is coming from a LAN segment that contains adapter DD-DD-DD-DD-DD-DD. There being no need to forward the frame to any of the other interfaces, the switch performs the filtering function by discarding the frame.
- There is an entry in the table, associating DD-DD-DD-DD-DD-DD with interface $y \neq x$. In this case, the frame needs to be forwarded to the LAN segment attached to interface *y*. The switch performs its forwarding function by putting the frame in an output buffer that precedes interface *y*.

Link-layer switches: Self-learning

- Switch tables are built automatically, dynamically and autonomously (i.e., switches are self-learning)
- Steps involved in switch table creation:
  1. The switch table is initially empty
  2. When a frame arrives, the switch stores in its switch table:
     - The MAC address in the frame's source address field
     - The interface from which the frame arrived
     - The current time
  3. If no frames are received from a MAC address within the **aging time**, the address entry is deleted

A switch has the wonderful property (particularly for the already-overworked network administrator) that its table is built automatically, dynamically, and autonomously— without any intervention from a network administrator or from a configuration protocol. In other words, switches are **self-learning**. This capability is accomplished as follows:

1. The switch table is initially empty.
2. For each incoming frame received on an interface, the switch stores in its table (1) the MAC address in the frame's *source address field*, (2) the interface from which the frame arrived, and (3) the current time. In this manner, the switch records in its table the LAN segment on which the sender resides. If every host in the LAN eventually sends a frame, then every host will eventually get recorded in the table.
3. The switch deletes an address in the table if no frames are received with that address as the source address after some period of time (the **aging time**). In this manner, if a PC is replaced by another PC (with a different adapter), the MAC address of the original PC will eventually be purged from the switch table.

Switches versus routers

Switches

- Store and forward: Link-layer devices
- Switch table
  - Self-learning based on flooding
  - Uses MAC addresses
- Pros
  - Plug-and-play (no need for manual configuration)
  - High forwarding rate (only process link-layer header)
- Cons
  - Restricted to a spanning-tree topology (to avoid cycles)
  - Switch table grows with number of hosts in subnet
  - Susceptible to broadcast storms (no TTL field)

Routers

- Store and forward: Network-layer devices
- Forwarding table:
  - Computed based on routing algorithms
  - Uses IP addresses
- Pros
  - Topology can have redundant paths
  - Firewall protection against layer-2 broadcast storms
- Cons
  - Not plug-and-play (configure IP addresses of connected hosts)
  - Larger processing time per packet (need to process layer-3 fields)

Modern SDN switches (e.g., using OpenFlow) use “match plus action” operations that can make use of both layer-2 frame information and layer-3 datagram information

University of Antwerp
Faculty of Science

58

Routers are store-and-forward packet switches that forward packets using network-layer addresses. Although a switch is also a store-and-forward packet switch, it is fundamentally different from a router in that it forwards packets using MAC addresses. Whereas a router is a layer-3 packet switch, a switch is a layer-2 packet switch. Recall that modern switches using the “match plus action” operation can be used to forward a layer-2 frame based on the frame's destination MAC address, as well as a layer-3 datagram using the datagram's destination IP address. Indeed, we saw that switches using the OpenFlow standard can perform generalized packet forwarding based on any of eleven different frame, datagram, and transport-layer header fields.

Even though switches and routers are fundamentally different, network administrators must often choose between them when installing an interconnection device. Given that both switches and routers are candidates for interconnection devices, what are the pros and cons of the two approaches?

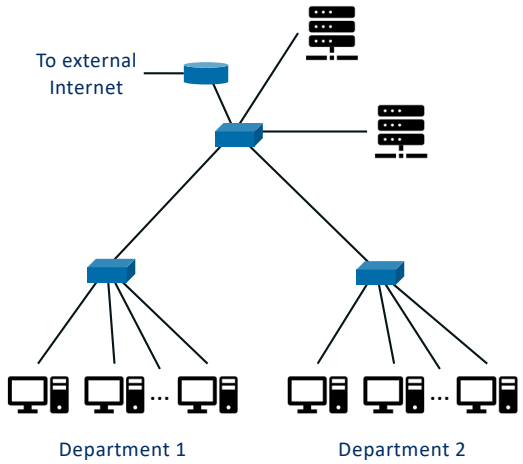
First consider the pros and cons of switches. As mentioned above, switches are plug-and-play, a property that is cherished by all the overworked network administrators of the world. Switches can also have relatively high filtering and forwarding rates— switches have to process frames only up through layer 2, whereas routers have to process datagrams up through layer 3. On the other hand, to prevent the cycling of broadcast frames, the active topology of a switched network is restricted to a spanning tree. Also, a large switched network would require large ARP tables in the hosts and routers and would generate substantial ARP traffic and processing. Furthermore, switches are susceptible to broadcast storms—if one host goes haywire and transmits an endless stream of Ethernet broadcast

frames, the switches will forward all of these frames, causing the entire network to collapse.

Now consider the pros and cons of routers. Because network addressing is often hierarchical (and not flat, as is MAC addressing), packets do not normally cycle through routers even when the network has redundant paths. Thus, packets are not restricted to a spanning tree and can use the best path between source and destination. Because routers do not have the spanning tree restriction, they have allowed the Internet to be built with a rich topology that includes, for example, multiple active links between Europe and North America. Another feature of routers is that they provide firewall protection against layer-2 broadcast storms. Perhaps the most significant drawback of routers, though, is that they are not plug-and-play—they and the hosts that connect to them need their IP addresses to be configured. Also, routers often have a larger per-packet processing time than switches, because they have to process up through the layer-3 fields.

Downsides of switch hierarchies

- LAN topology with hierarchy of switches has several downsides:
  - **Lack of traffic isolation:** Broadcast traffic (ARP, DHCP, frames of unknown destination) traverses entire hierarchy
  - **Inefficient use of switches**: While a large switch could connect all users, it wouldn't provide any traffic isolation
  - **Managing users**: If a users moves between departments, physical cabling needs to be changed
- **Solution**: virtual local area networks (VLANs)!



University of Antwerp
Faculty of Science

59

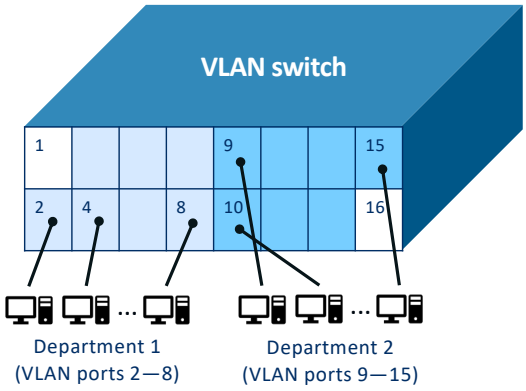
Modern institutional LANs are often configured hierarchically, with each workgroup (department) having its own switched LAN connected to the switched LANs of other groups via a switch hierarchy. While such a configuration works well in an ideal world, the real world is often far from ideal. Three drawbacks can be identified in this configuration:

- ***Lack of traffic isolation.*** Although the hierarchy localizes group traffic to within a single switch, broadcast traffic (e.g., frames carrying ARP and DHCP messages or frames whose destination has not yet been learned by a self-learning switch) must still traverse the entire institutional network. Limiting the scope of such broadcast traffic would improve LAN performance. Perhaps more importantly, it also may be desirable to limit LAN broadcast traffic for security/privacy reasons. For example, if one group contains the company's executive management team and another group contains disgruntled employees running Wireshark packet sniffers, the network manager may well prefer that the executives' traffic never even reaches employee hosts. This type of isolation could be provided by replacing the center switch in the figure with a router.
- ***Inefficient use of switches.*** If instead of three groups, the institution had 10 groups, then 10 first-level switches would be required. If each group were small, say less than 10 people, then a single 96-port switch would likely be large enough to accommodate everyone, but this single switch would not provide traffic isolation.
- ***Managing users.*** If an employee moves between groups, the physical cabling must be changed to connect the employee to a different switch. Employees belonging to two groups make the problem even harder.

Fortunately, each of these difficulties can be handled by a switch that supports **virtual local area networks** (**VLANs**).

Virtual local area networks (VLANs)

- Multiple *virtual* LANs defined over a single *physical* LAN infrastructure
- In **port-based VLANs**, each switch port (interface) is manually assigned to a VLAN
- Hosts in a VLAN act as if connected to each other via a physical switch
- Each VLAN forms a broadcast domain (i.e., broadcast traffic is not forwarded across VLANs)
- Sending traffic between VLANs can be done by defining a port as belonging to both VLANs and connecting it to a router



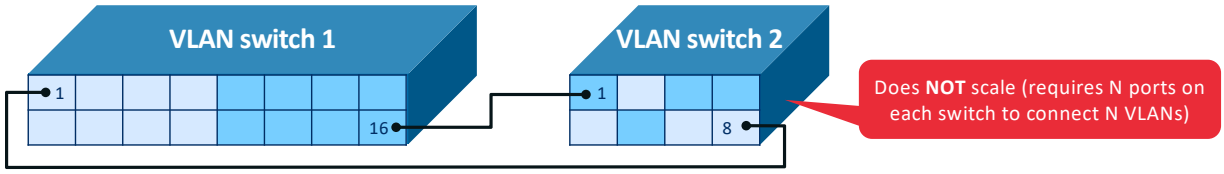
University of Antwerp
Faculty of Science

As the name suggests, a switch that supports VLANs allows multiple *virtual* local area networks to be defined over a single *physical* local area network infrastructure. Hosts within a VLAN communicate with each other as if they (and no other hosts) were connected to the switch. In a port-based VLAN, the switch's ports (interfaces) are divided into groups by the network manager. Each group constitutes a VLAN, with the ports in each VLAN forming a broadcast domain (i.e., broadcast traffic from one port can only reach other ports in the group).

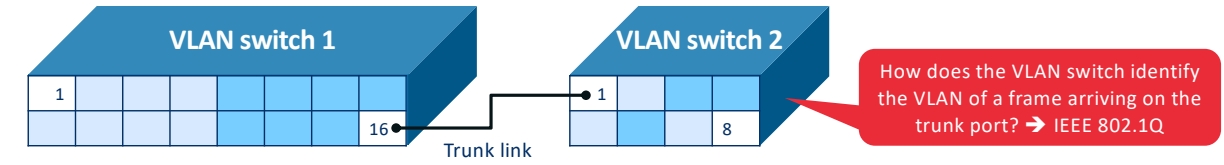
But by completely isolating the two VLANs, we have introduced a new difficulty! How can traffic from one department be sent to the other? One way to handle this would be to connect a VLAN switch port (e.g., port 1 in the figure) to an external router and configure that port to belong to both VLANs. In this case, even though the departments share the same physical switch, the logical configuration would look as if the departments had separate switches connected via a router. An IP datagram going from one department to the other would first cross the department 1 VLAN to reach the router and then be forwarded by the router back over the department 2 VLAN to the destination host. Fortunately, switch vendors make such configurations easy for the network manager by building a single device that contains both a VLAN switch *and* a router, so a separate external router is not needed.

Connecting multiple physical networks in a single VLAN

Solution 1: Directly connect port belonging to the same VLAN on both switches with a cable



Solution 2: VLAN trunking (A special port on each switch is configured to interconnect the two VLAN switches)



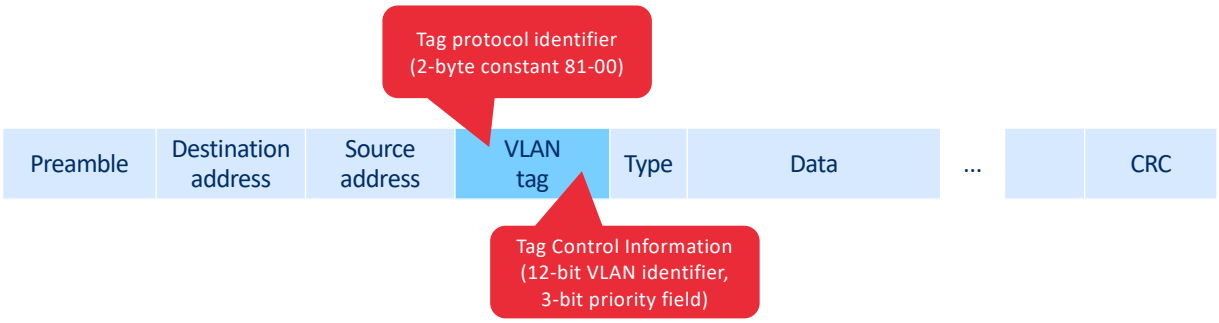
University of Antwerp
Faculty of Science

61

Let's now assume the departments are split across different buildings. The members of each department need network access, but also want to belong to the same VLAN. The figures show a second 8-port switch, where the switch ports have been defined as belonging to the two VLANs, as needed. But how should these two switches be interconnected? One easy solution would be to define a port belonging to the first VLAN on each switch (similarly for the second VLAN) and to connect these ports to each other, as shown in the top figure. This solution doesn't scale, however, since *N* VLANS would require *N* ports on each switch simply to interconnect the two switches.

A more scalable approach to interconnecting VLAN switches is known as **VLAN trunking**. In the VLAN trunking approach shown in the bottom figure, a special port on each switch (port 16 on the left switch and port 1 on the right switch) is configured as a trunk port to interconnect the two VLAN switches. The trunk port belongs to all VLANs, and frames sent to any VLAN are forwarded over the trunk link to the other switch. But this raises yet another question: How does a switch know that a frame arriving on a trunk port belongs to a particular VLAN? The IEEE has defined an extended Ethernet frame format, 802.1Q, for frames crossing a VLAN trunk.

IEEE 802.1Q: extended Ethernet frame format for VLAN trunking



- Four-byte VLAN tag consists of two-byte tag protocol identifier and two-byte tag control information
- VLAN tag is added to the frame by the sending switch of the VLAN trunk link
- VLAN tag is removed by the receiving switch of the VLAN trunk link

The 802.1Q frame consists of the standard Ethernet frame with a four-byte **VLAN tag** added into the header that carries the identity of the VLAN to which the frame belongs. The VLAN tag is added into a frame by the switch at the sending side of a VLAN trunk, parsed, and removed by the switch at the receiving side of the trunk. The VLAN tag itself consists of a 2-byte Tag Protocol Identifier (TPID) field (with a fixed hexadecimal value of 81-00), a 2-byte Tag Control Information field that contains a 12-bit VLAN identifier field, and a 3-bit priority field that is similar in intent to the IP datagram TOS field.

Summary

University of Antwerp
Faculty of Science

Summary

- Link-layer is responsible for transferring data across a single link
- **Functionality** can include framing, link access, reliability, and error detection/correction
- Different methods can be used for **error detection**: parity checks, checksums, and cyclic redundancy checks (CRC)
- Multiple access protocols are used to avoid collisions on broadcast links
  - Channel partitioning protocols (FDMA, TDMA, CDMA)
  - Random access protocols (ALOHA, slotted ALOHA, CSMA/CD, CSMA/CA): Used by Ethernet and Wi-Fi
  - Taking-turns protocols (polling, token-passing): Combines advantages of channel partitioning and random access
- Local area networks (LANs) make use of MAC addresses, ARP, and self-learning switches to forward frames
  - VLAN switches can define multiple virtual LANs on a single physical switch for broadcast and traffic isolation
- The contents of this part is covered in the **book** in Sections 6.1, 6.2, 6.3, 6.4, 7.2.1, and 7.3.2

End of Part 5

University of Antwerp
Faculty of Science